

Lecture 06:

Optimization



Markus Hohle

University of California, Berkeley

Machine Learning Algorithms

MSSE 277B, 3 Units



Lecture 1: Course Overview and Introduction to Machine Learning

Lecture 2: Bayesian Methods in Machine Learning

classic ML tools & algorithms

Lecture 3: Dimensionality Reduction: Principal Component Analysis

Lecture 4: Linear and Non-linear Regression and Classification

Lecture 5: Unsupervised Learning: K-Means, GMM, Trees

Lecture 6: Adaptive Learning and Gradient Descent Optimization Algorithms

Lecture 7: Introduction to Artificial Neural Networks - The Perceptron

ANNs/AI/Deep Learning

Lecture 8: Introduction to Artificial Neural Networks - Building Multiple Dense Layers

Lecture 9: Convolutional Neural Networks (CNNs) - Part I

Lecture 10: CNNs - Part II

Lecture 11: Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTMs)

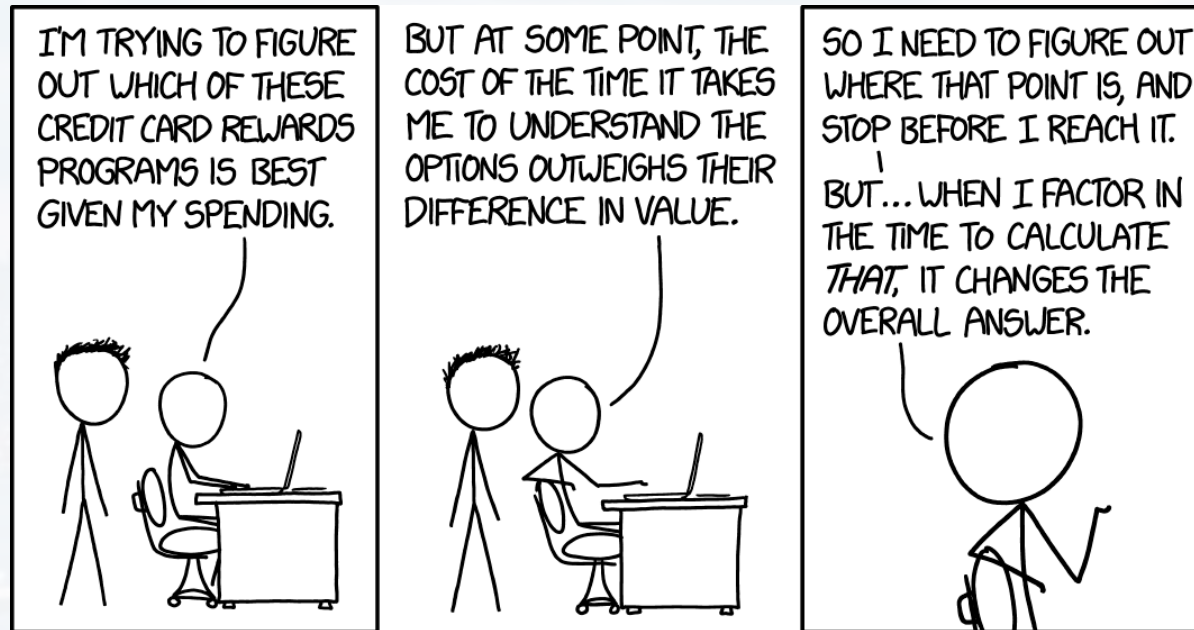
Lecture 12: Combining LSTMs and CNNs

Lecture 13: Running Models on GPUs and Parallel Processing

Lecture 14: Transformer

Lecture 15: GNN

Lecture 16: Project Presentations



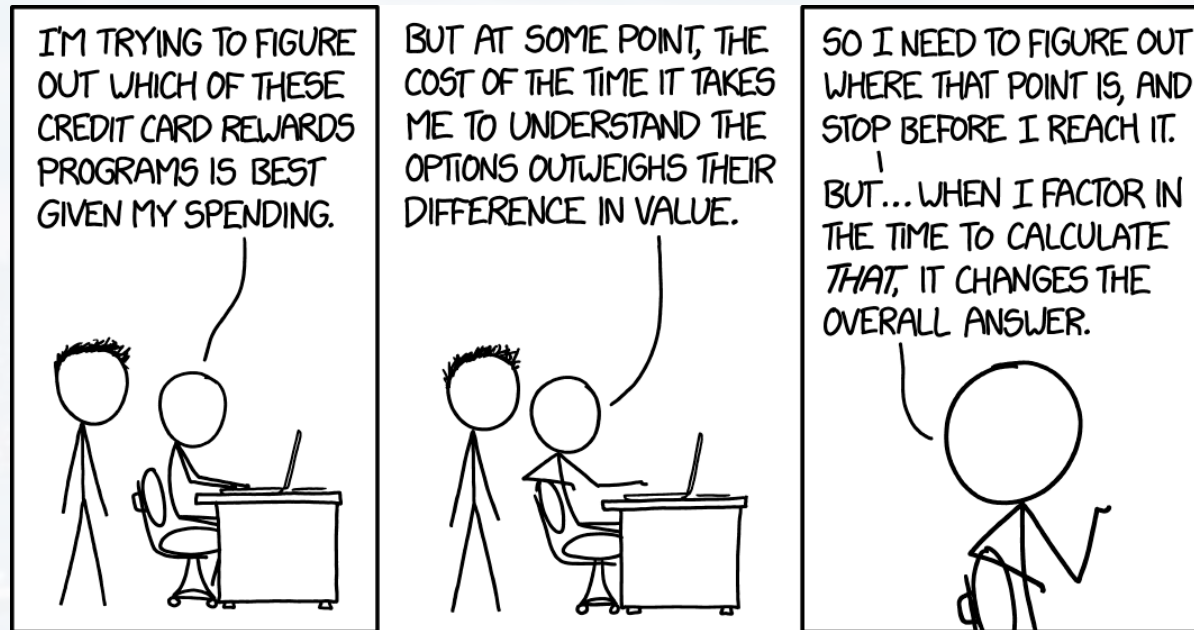
Outline

- The Problem

- Gradient Descent

- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning

- Simulated Annealing



Outline

- The Problem

- Gradient Descent

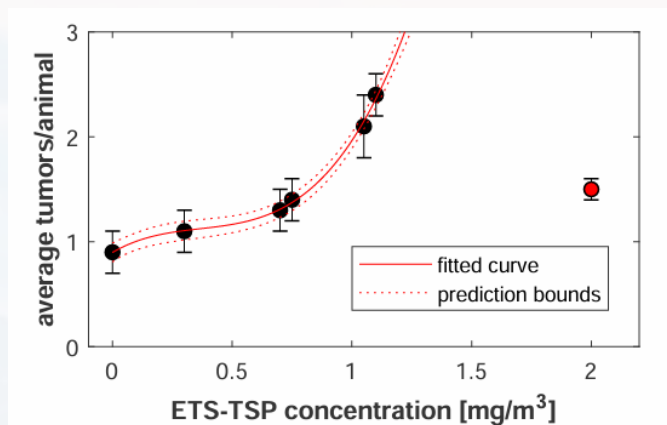
- Vanilla
- Learning Rate Schedule
- Momentum
- L1 and L2
- More Finetuning

- Simulated Annealing



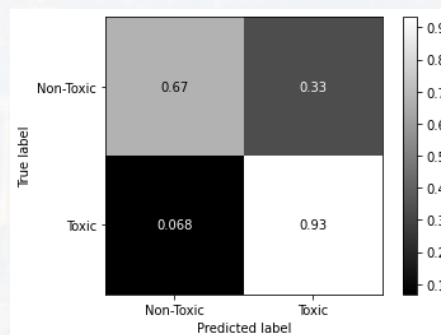
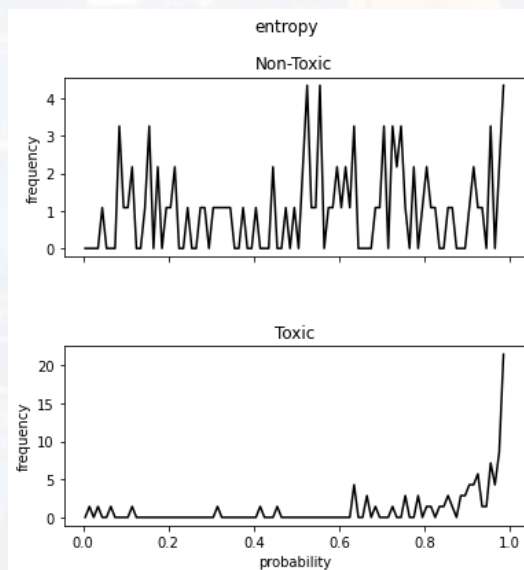
Any algorithm needs a “goal” aka **objective function** that has to be *optimized* (finding an **extreme**)

regression, e. g.
curve fitting



minimize:
$$\chi_{red}^2 = \frac{1}{N - p - 1} \sum_{i=1}^N \frac{(\hat{y}(model)_i - y_i)^2}{\sigma_i^2}$$

classification



maximize: accuracy

minimize: cross entropy

$$S = - \sum_i p(true)_i \cdot \ln p(model)_i$$



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

Often, the extreme of the objective function is subject to **constraints**

cross entropy

$$S = - \sum_i p(\text{true})_i \cdot \ln p(\text{model})_i$$

constrain: $\sum_i p_i = 1$

→ Lagrangian Multipliers and variational calculus

→ mathematically:

Free Energy like term = Energy like term – Entropy term

examples:

- Evidence **Lower Bound**
- Lasso method (linear regression)
- actual energy → Boltzmann distribution

etc

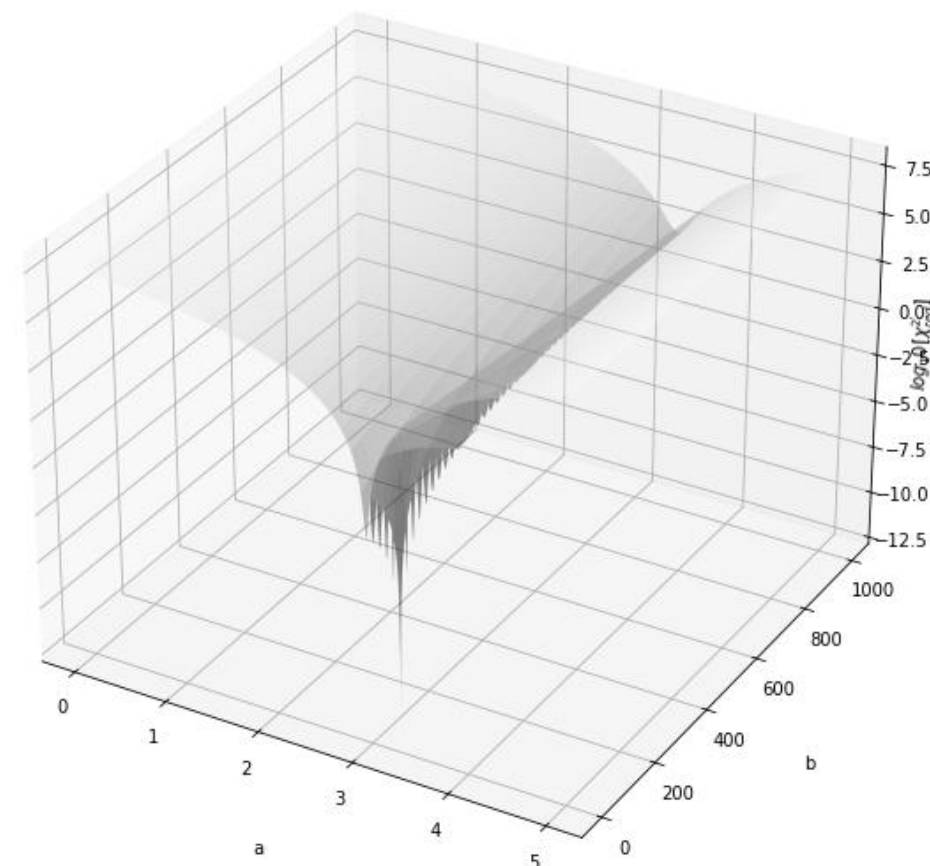


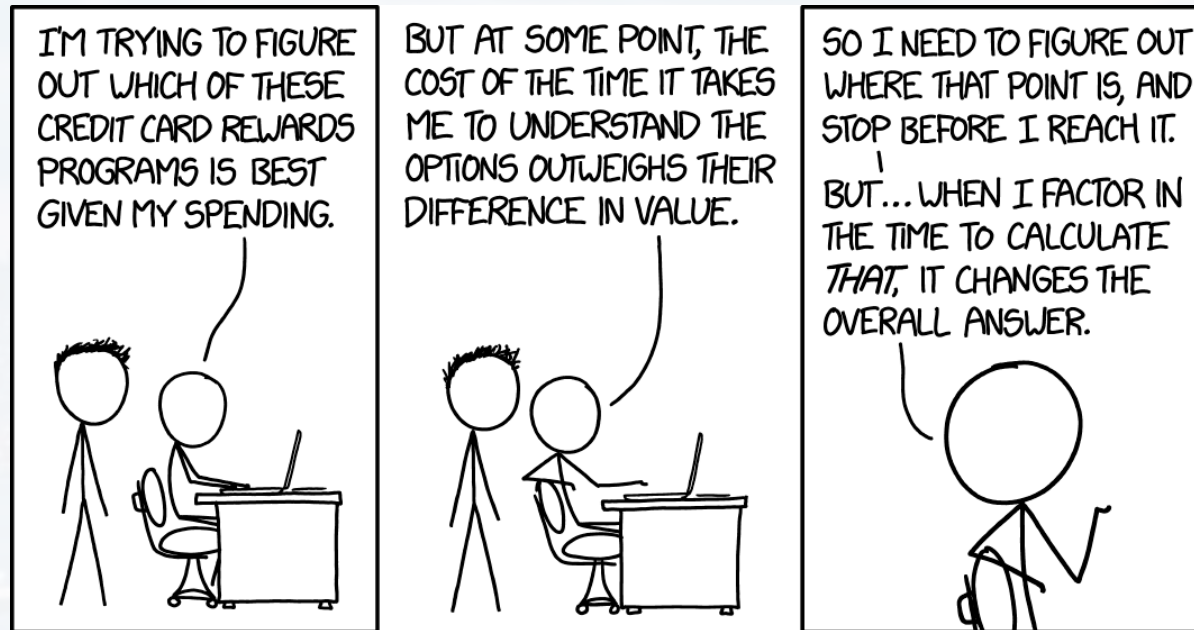
Any algorithm needs a “goal” aka **objective function** that has to be *optimized* (finding an **extreme**)

These functions are very complicated, **not analytical (= no mathematical equation)** at all

two most common approaches:

- gradient descent
- simulated annealing





Outline

- The Problem

- **Gradient Descent**

- Vanilla

- Learning Rate Schedule

- Momentum

- L1 and L2

- More Finetuning

- Simulated Annealing



main application: **ANN!**



ChatGPT,
Spring 2025

ChatGPT ▾

✦ Get Plus ✕

How likely is the "Terminator Scenario"?

The "Terminator Scenario" — meaning a future where AI becomes self-aware, turns hostile, and deliberately tries to exterminate humanity like in The Terminator — is considered **extremely unlikely** by mainstream AI researchers.





main application: **ANN!**



US ▾

The Guardian

News Opinion Sport Culture Lifestyle ☰

UK US politics World Climate crisis Middle East Ukraine Football Newsletters Business Environment UK politics Science **Tech** Global development Obituaries

AI (artificial intelligence)

AI could kill all humans in next decade, warn experts: but how seriously should we take them?

Alarming warnings from industry insiders increase pressure for curbs on artificial superintelligence

Most viewed

 Concern mounts over slow-moving, planetary-scale 'Kelvin' wave heading for California

 Texas Republican called for public execution of



one of many
headlines,
Fall 2026

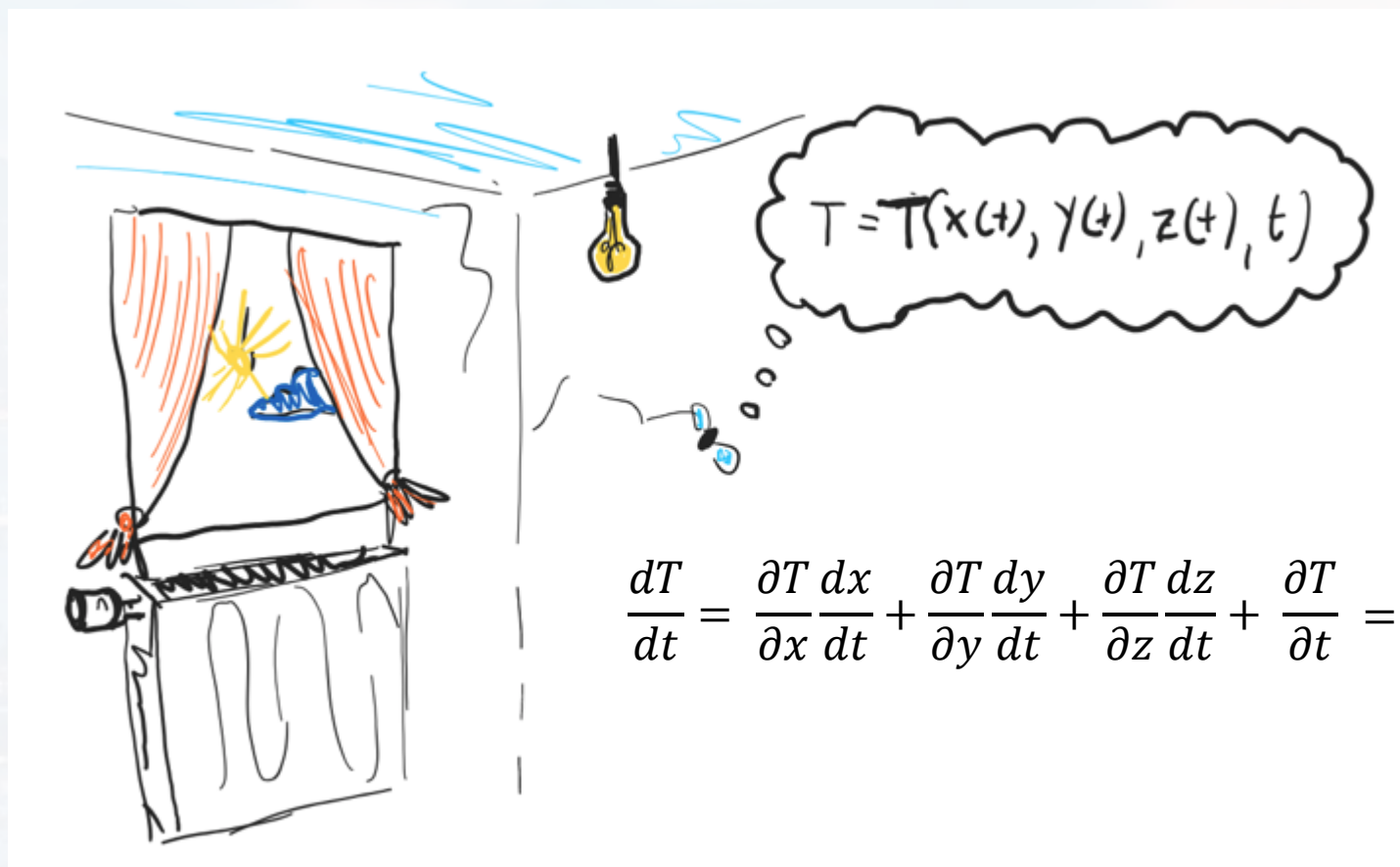


Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

→ extreme of an objective function

gradient
descent

temperature profile in space and time



T : temperature

$$\frac{dT}{dt} = \frac{\partial T}{\partial x} \frac{dx}{dt} + \frac{\partial T}{\partial y} \frac{dy}{dt} + \frac{\partial T}{\partial z} \frac{dz}{dt} + \frac{\partial T}{\partial t} = \begin{pmatrix} \frac{\partial T}{\partial x} \\ \frac{\partial T}{\partial y} \\ \frac{\partial T}{\partial z} \end{pmatrix} \circ \begin{pmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \\ \frac{dz}{dt} \end{pmatrix} + \frac{\partial T}{\partial t}$$

$$= \text{grad}(T) \circ \vec{v}$$

If T doesn't change
with time!



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

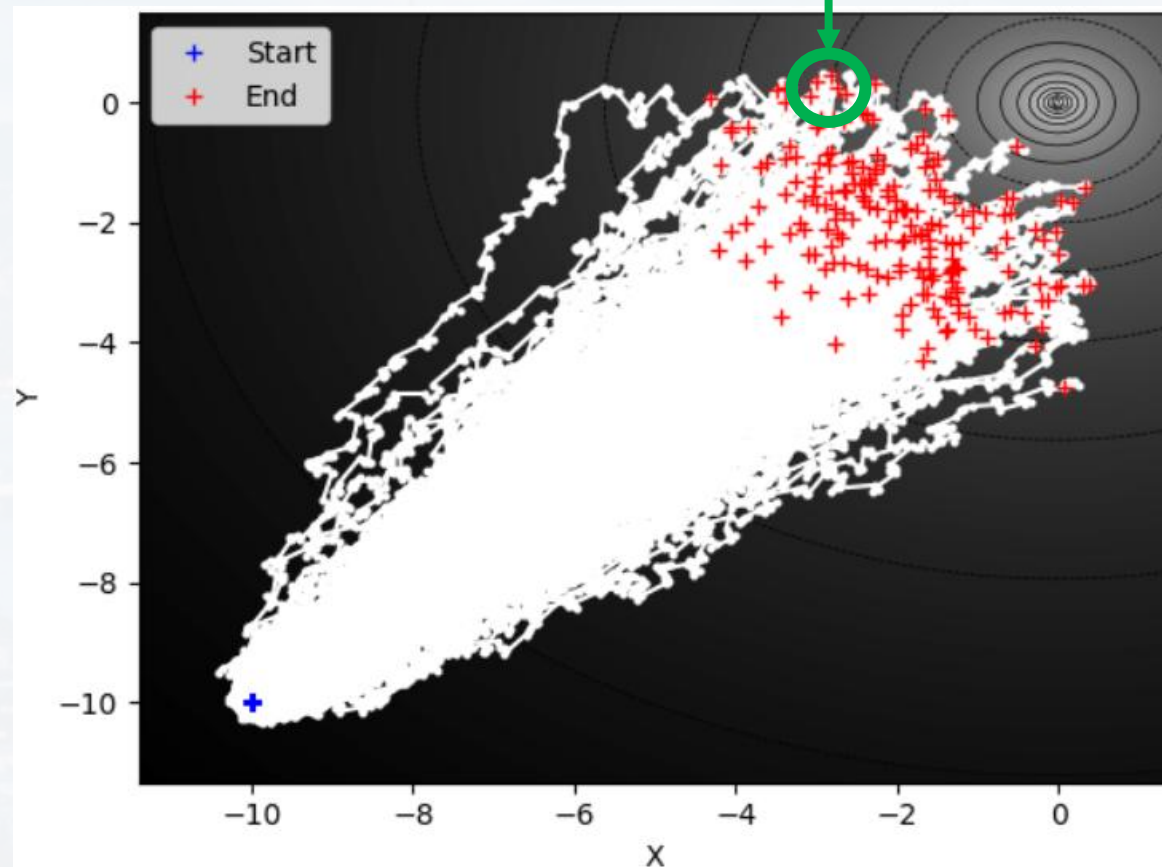
→ extreme of an objective function

gradient
descent

concentration profile in space and time

E. Coli

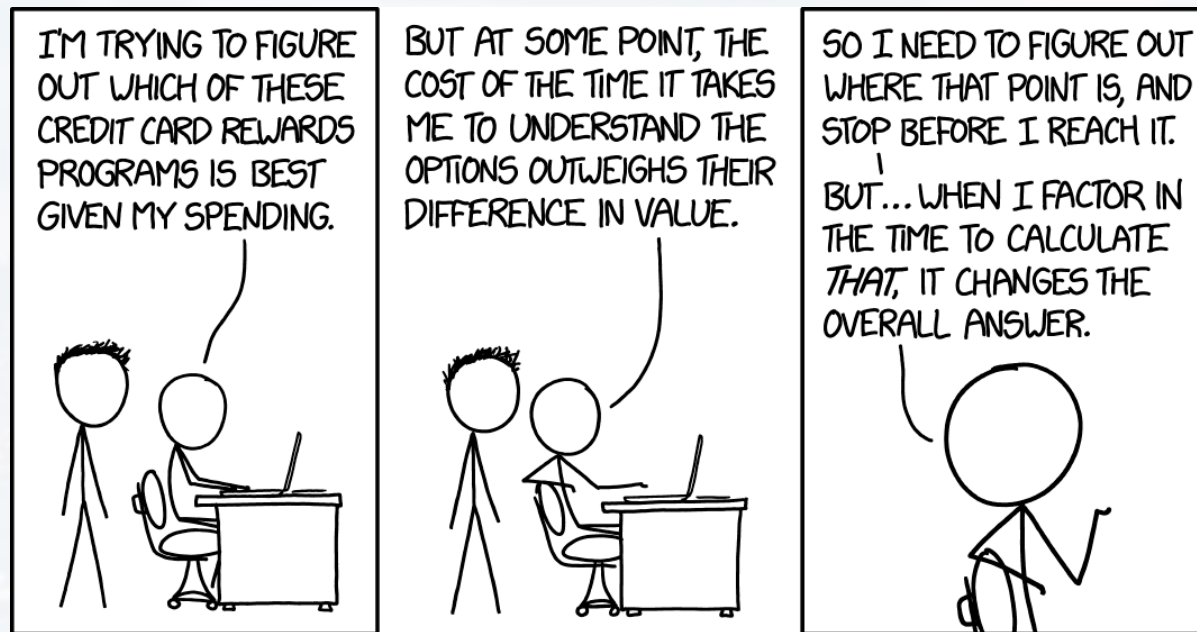
c: concentration



$$\frac{dc}{dt} = \frac{\partial c}{\partial x} \frac{dx}{dt} + \frac{\partial c}{\partial y} \frac{dy}{dt} + \frac{\partial c}{\partial z} \frac{dz}{dt} + \frac{\partial c}{\partial t}$$

$$= \begin{pmatrix} \frac{\partial c}{\partial x} \\ \frac{\partial c}{\partial y} \\ \frac{\partial c}{\partial z} \end{pmatrix} \circ \begin{pmatrix} \frac{dx}{dt} \\ \frac{dy}{dt} \\ \frac{dz}{dt} \end{pmatrix} + \frac{\partial c}{\partial t}$$

$$= \text{grad}(c) \circ \vec{v} \quad \text{If } c \text{ doesn't change with time!}$$



Outline

- The Problem

- **Gradient Descent**

- Vanilla

- Learning Rate Schedule

- Momentum

- L1 and L2

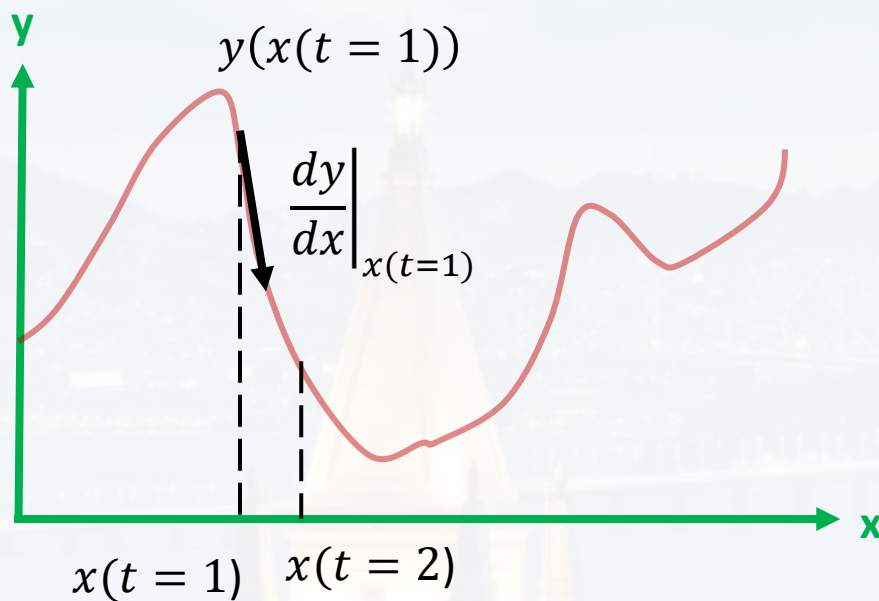
- More Finetuning

- Simulated Annealing



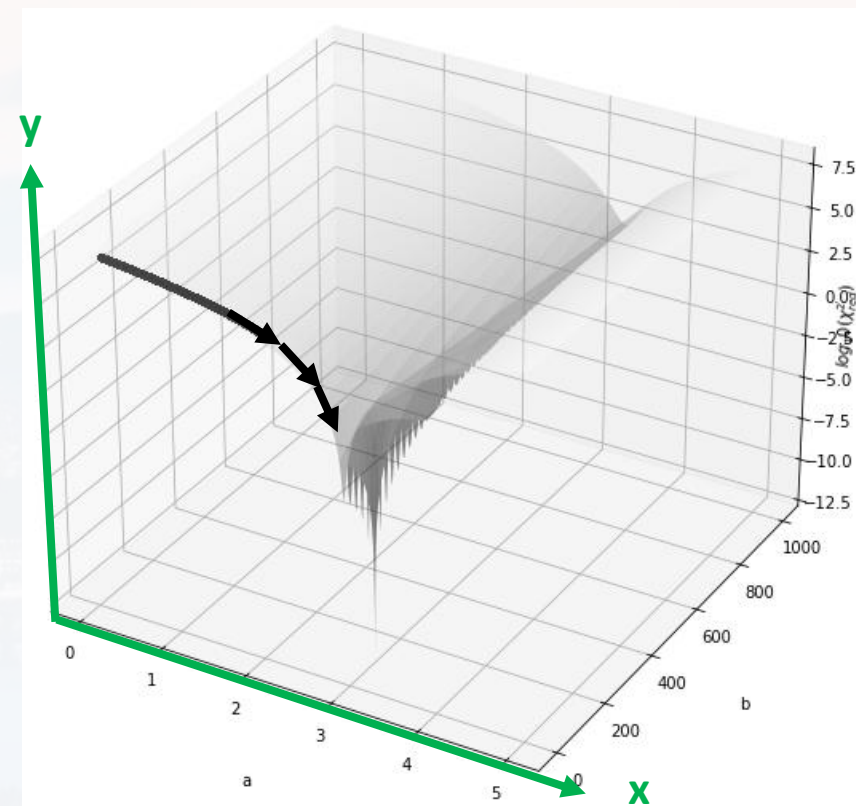
Vanilla

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



$$x(t=2) = x(t=1) - \epsilon \left. \frac{dy}{dx} \right|_{x(t=1)}$$

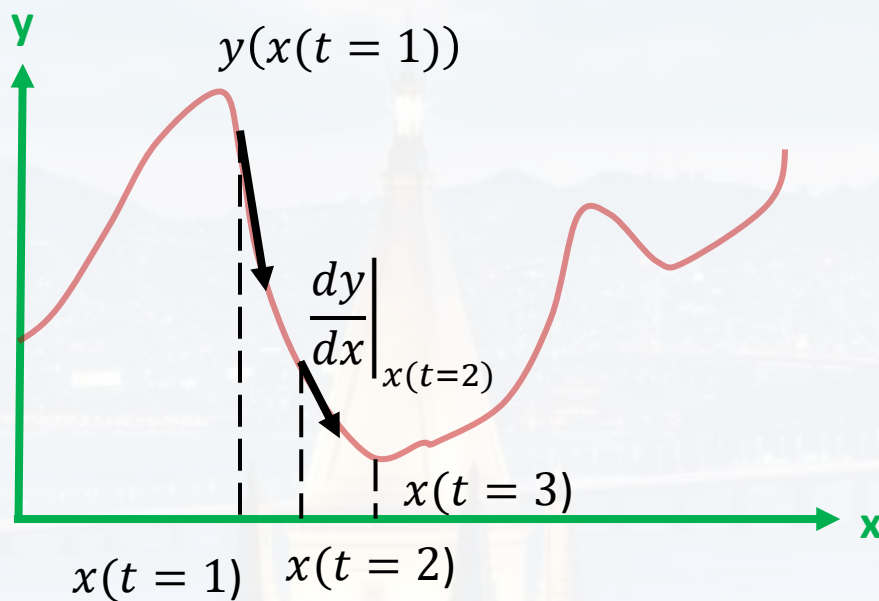
$$\epsilon > 0$$



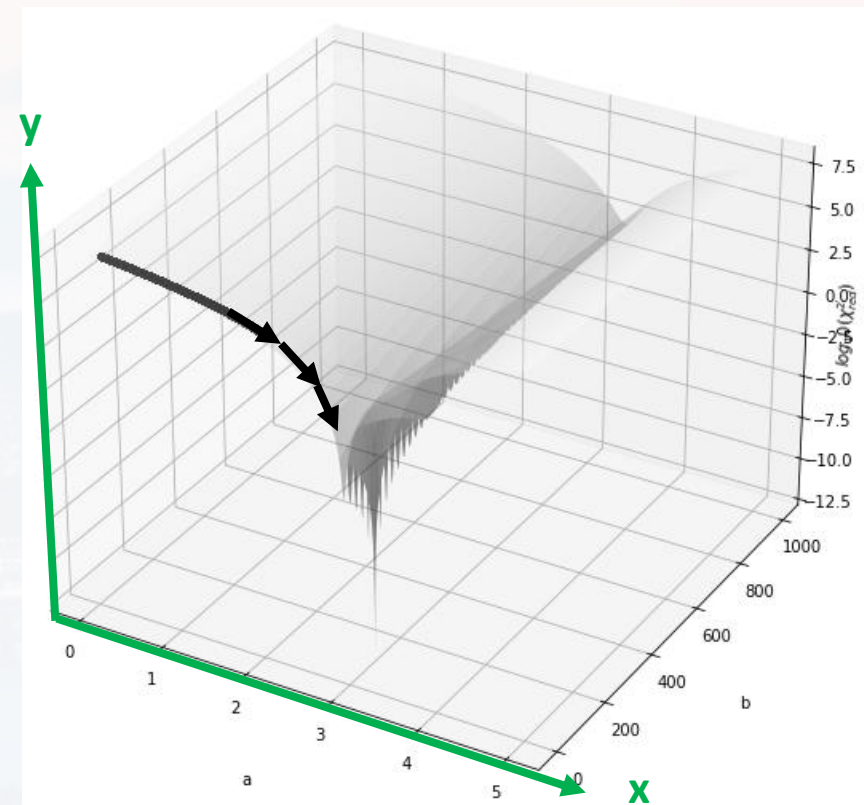


Vanilla

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



$$x(t=3) = x(t=2) - \epsilon \left. \frac{dy}{dx} \right|_{x(t=2)}$$

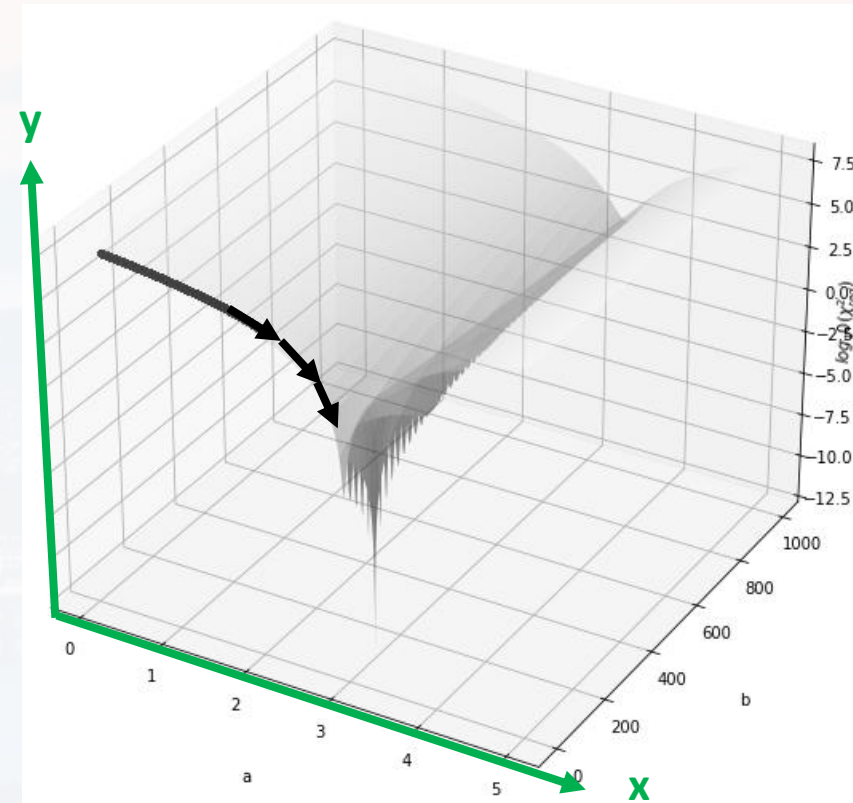
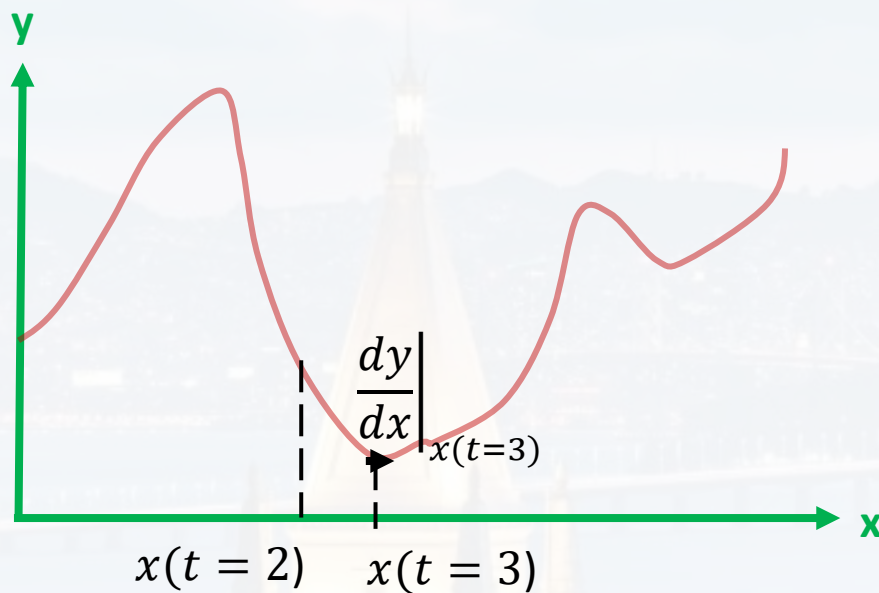


$$\epsilon > 0$$



Vanilla

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



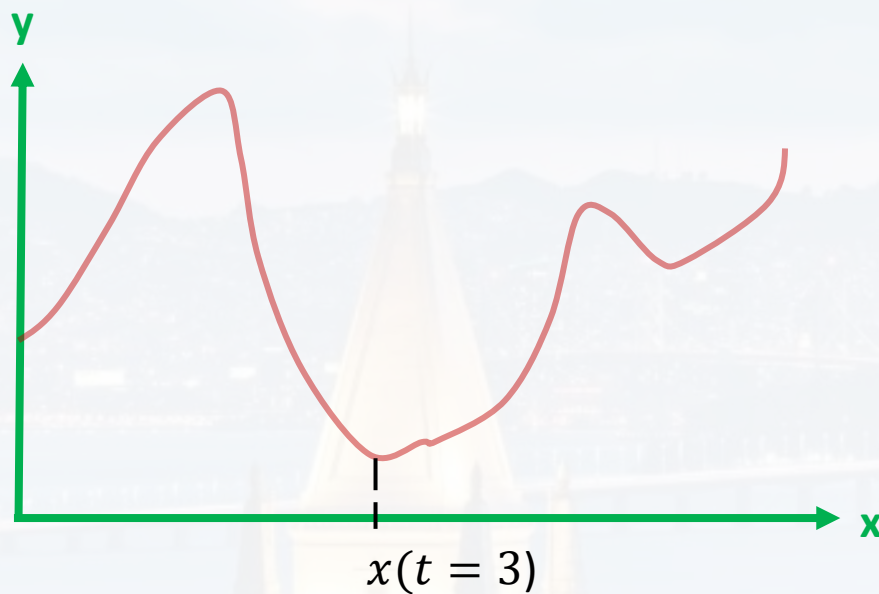
$$x(t=4) = x(t=3) - \varepsilon \left. \frac{dy}{dx} \right|_{x(t=3)}$$

$$\varepsilon > 0$$



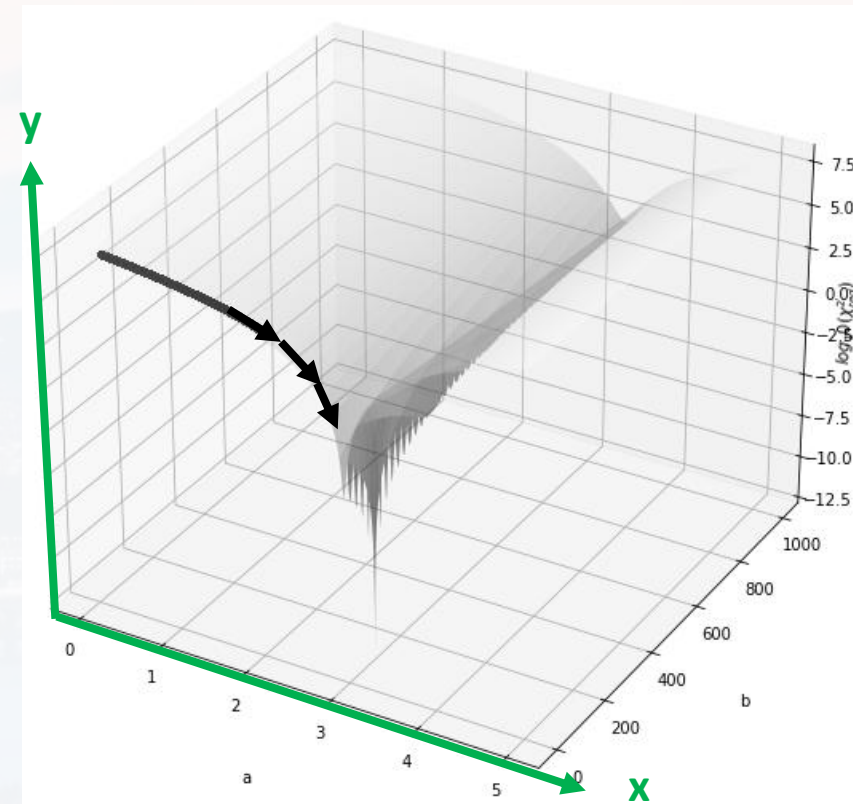
Vanilla

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



$$x(t=4) = x(t=3) - \epsilon \left. \frac{dy}{dx} \right|_{x(t=3)}$$

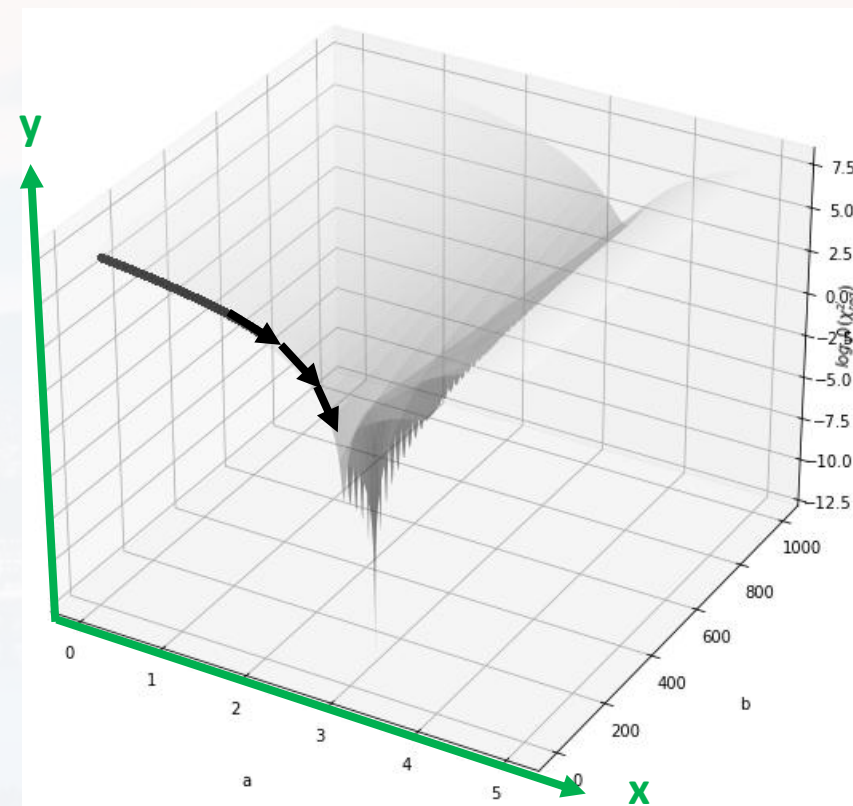
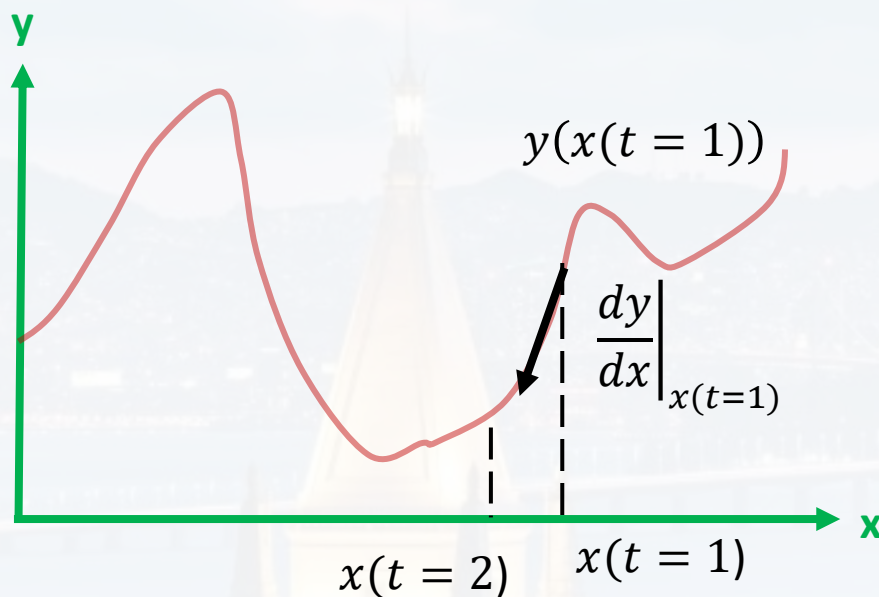
$$\epsilon > 0$$





Vanilla

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$



$$x(t=2) = x(t=1) - \epsilon \left. \frac{dy}{dx} \right|_{x(t=1)}$$

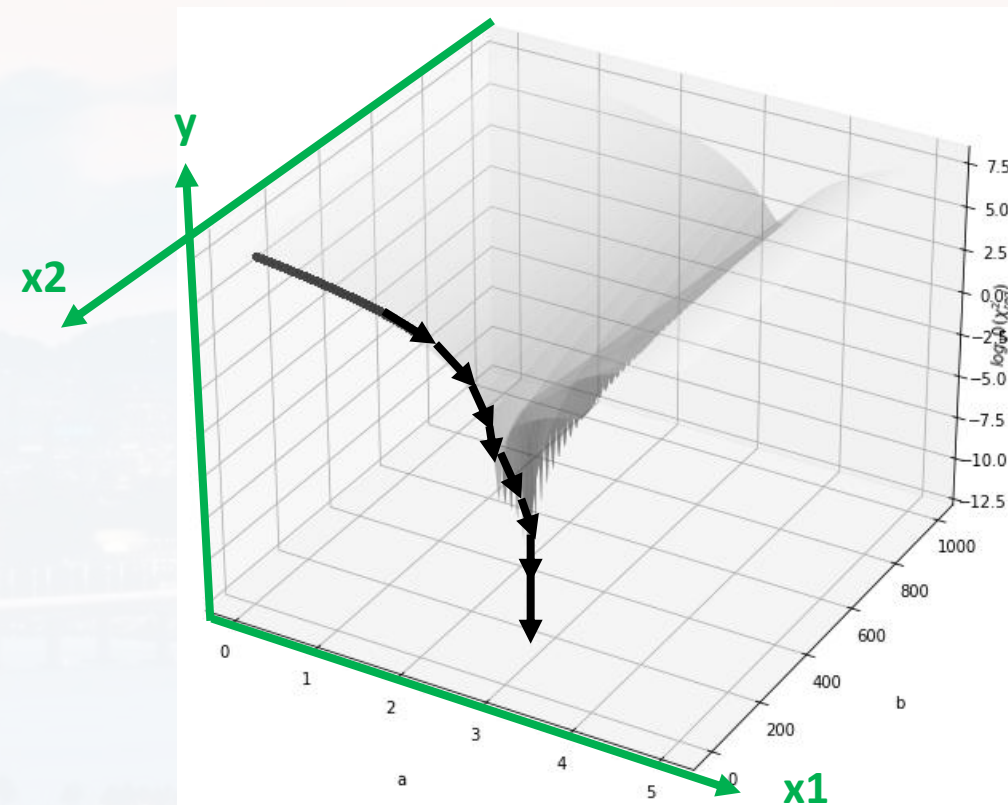
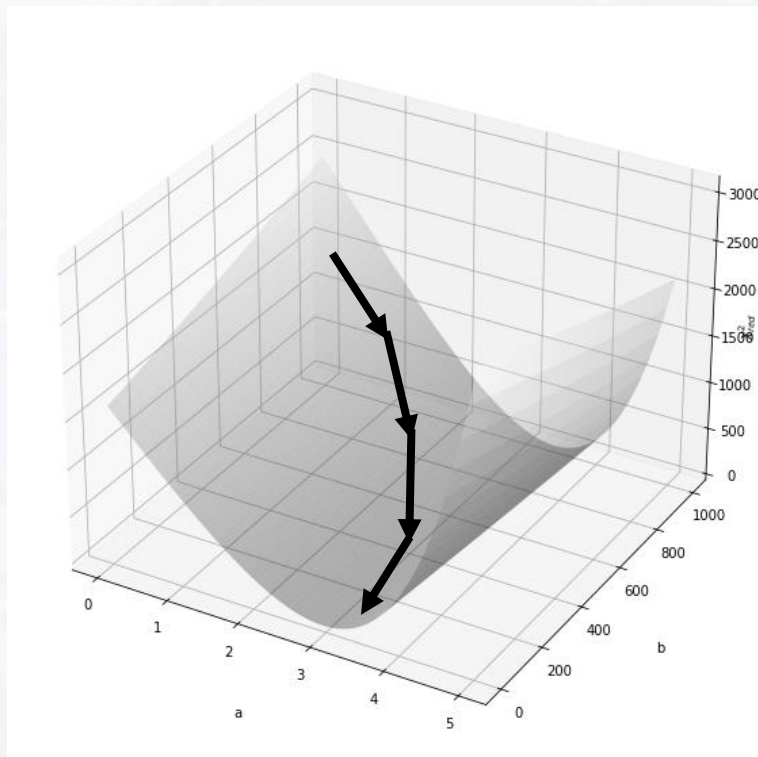
$$\epsilon > 0$$



Vanilla

$$\left. \frac{\partial y}{\partial x_1} \right|_{x_1^*; x_2^*} \approx \frac{y(x_1^* + \Delta x_1, x_2^*) - y(x_1^* - \Delta x_1, x_2^*)}{2\Delta x_1}$$

$$\left. \frac{\partial y}{\partial x_2} \right|_{x_1^*; x_2^*} \approx \frac{y(x_1^*, x_2^* + \Delta x_2) - y(x_1^*, x_2^* - \Delta x_2)}{2\Delta x_2}$$





Vanilla

$$\left. \frac{\partial y}{\partial x_1} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(x_1^* + \Delta x_1, x_2^*, \dots, x_N^*) - y(x_1^* - \Delta x_1, x_2^*, \dots, x_N^*)}{2\Delta x_1}$$

$$\left. \frac{\partial y}{\partial x_2} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(x_1^*, x_2^* + \Delta x_2, \dots, x_N^*) - y(x_1^*, x_2^* - \Delta x_2, \dots, x_N^*)}{2\Delta x_2}$$

⋮

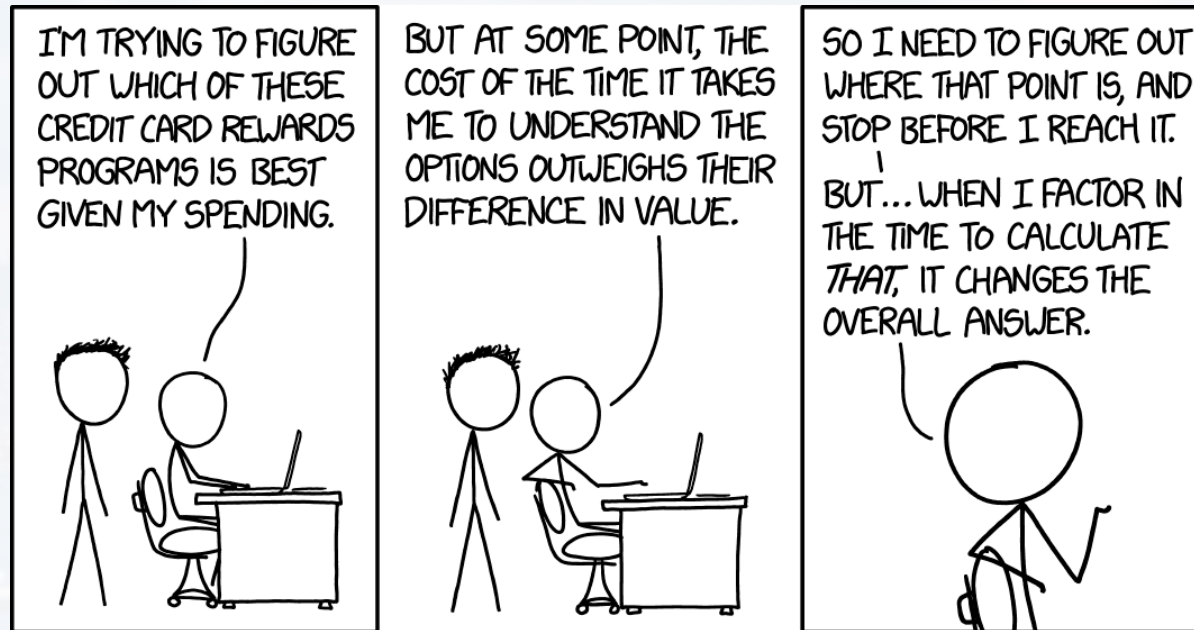
$$\left. \frac{\partial y}{\partial x_i} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(\dots, x_i^* + \Delta x_i, \dots, x_N^*) - y(\dots, x_i^* - \Delta x_i, \dots, x_N^*)}{2\Delta x_i}$$

⋮

$$\left. \frac{\partial y}{\partial x_N} \right|_{x_1^*; x_2^*; \dots; x_N^*} \approx \frac{y(x_1^*, x_2^*, \dots, x_N^* + \Delta x_N) - y(x_1^*, x_2^*, \dots, x_N^* - \Delta x_N)}{2\Delta x_N}$$

$$\begin{pmatrix} \left. \frac{\partial y}{\partial x_1} \right|_{x_1^*; x_2^*; \dots; x_N^*} \\ \vdots \\ \left. \frac{\partial y}{\partial x_i} \right|_{x_1^*; x_2^*; \dots; x_N^*} \\ \vdots \\ \left. \frac{\partial y}{\partial x_N} \right|_{x_1^*; x_2^*; \dots; x_N^*} \end{pmatrix}$$

= $grad(y)_x$
gradient of y wrt x



Outline

- The Problem

- **Gradient Descent**

- Vanilla

- Learning Rate Schedule

- Momentum

- L1 and L2

- More Finetuning

- Simulated Annealing



Learning Rate Schedule

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

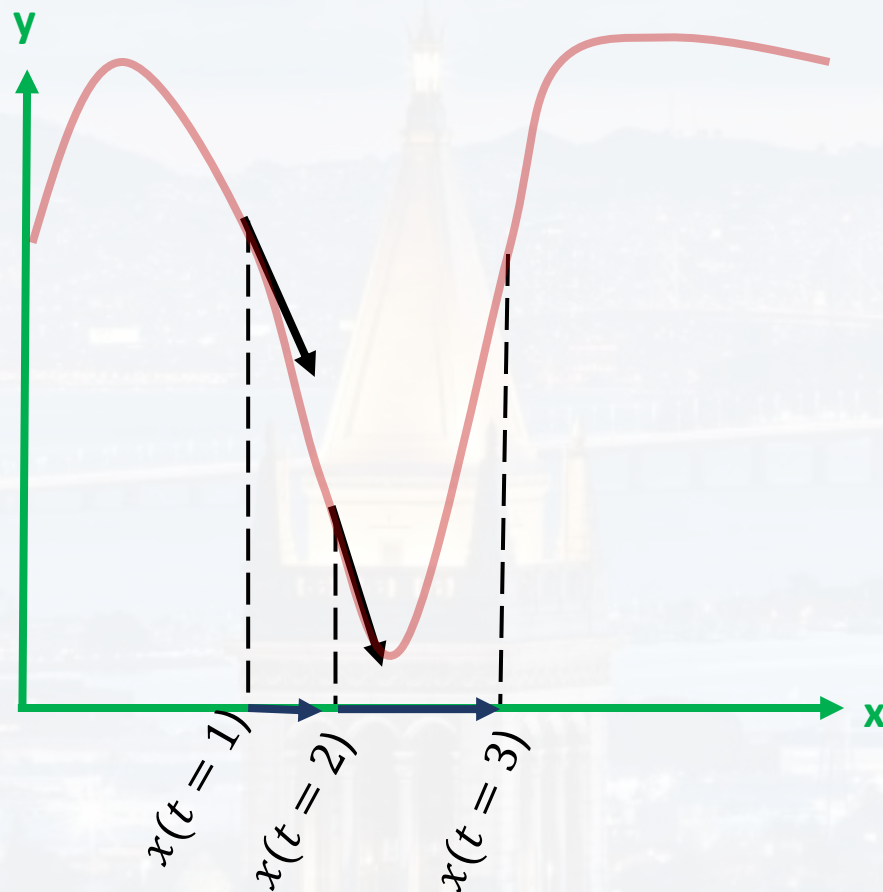
$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

called *learning rate*

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is





Learning Rate Schedule

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

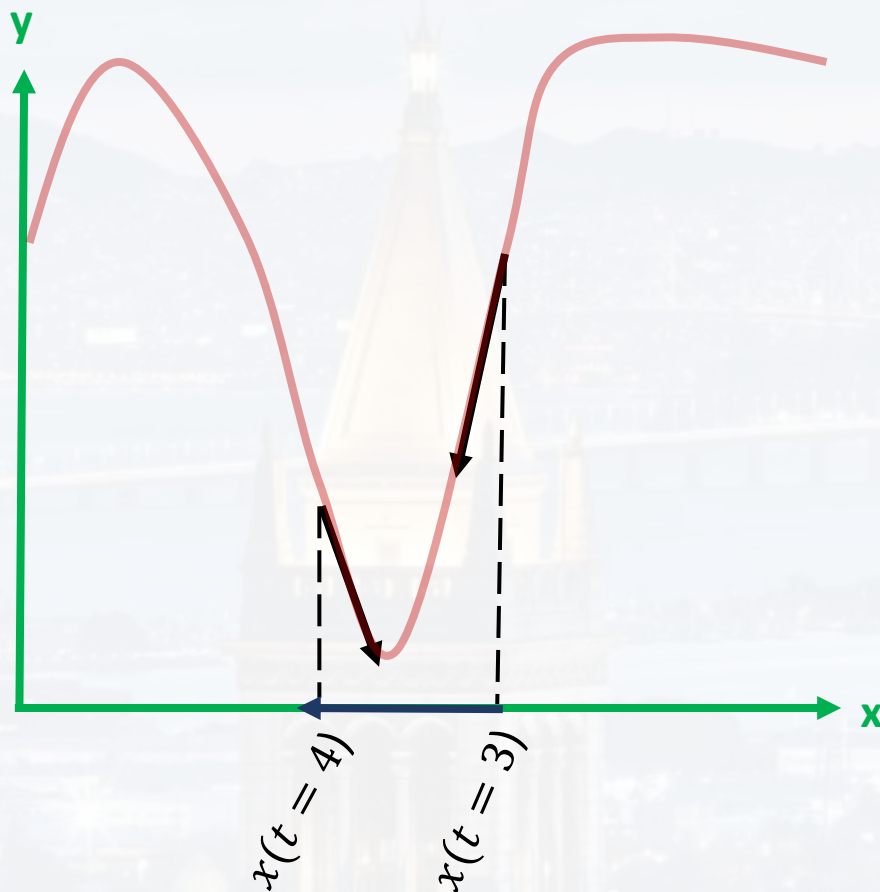
$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

called *learning rate*

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large
the leap Δx is





Learning Rate Schedule

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

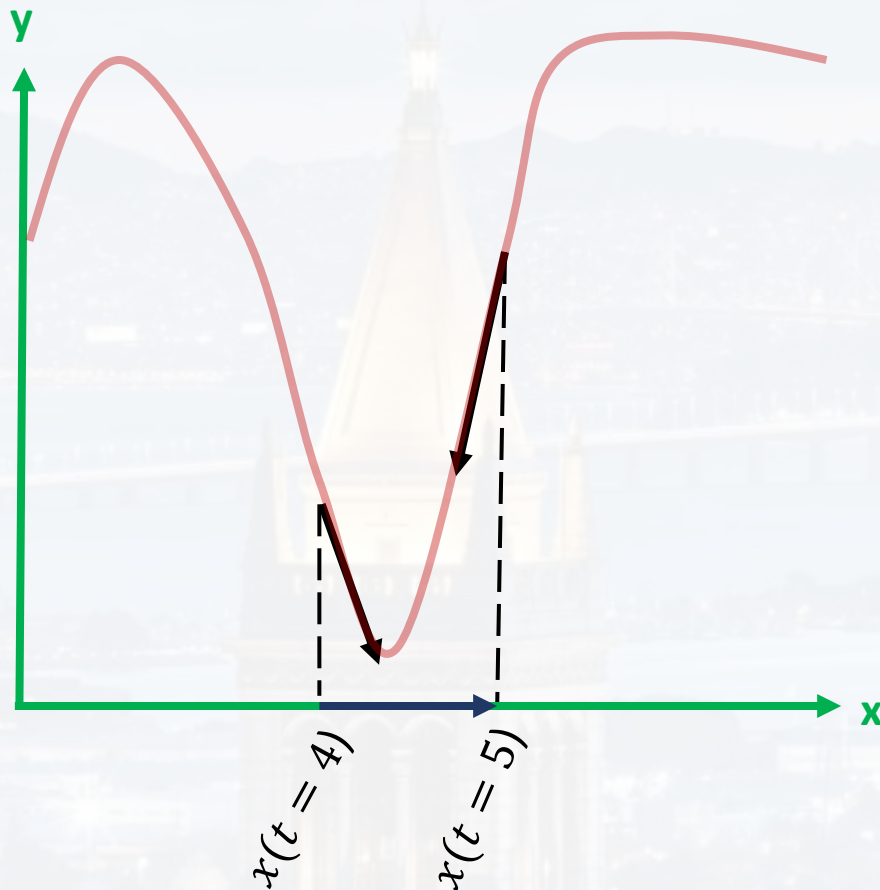
called *learning rate*

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large the leap Δx is

... and so on...

→ smaller ϵ ?





Learning Rate Schedule

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

$$\epsilon > 0$$

called *learning rate*

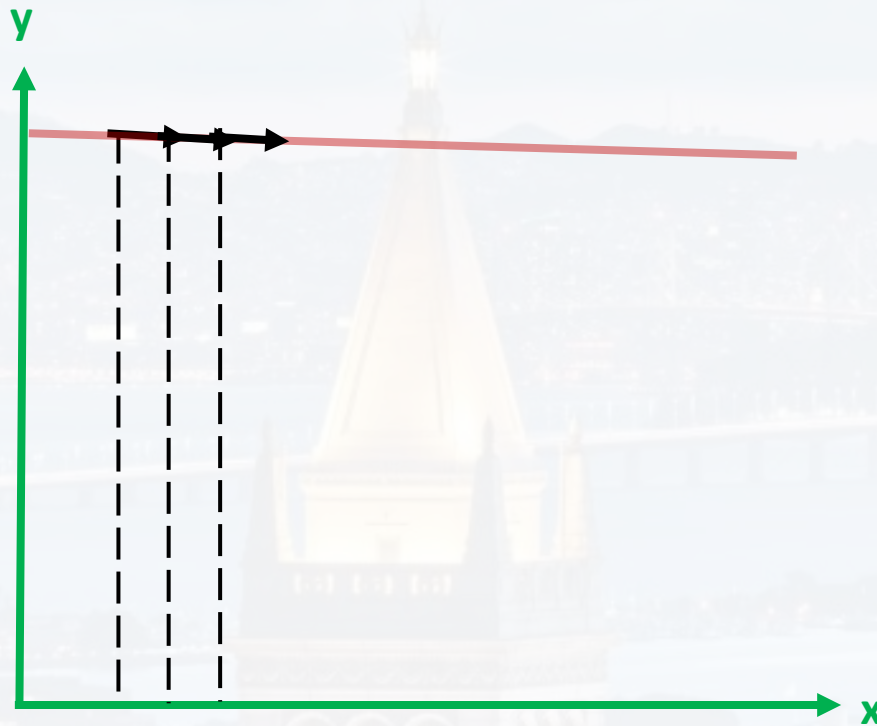
$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large the leap Δx is

... and so on...

→ smaller ϵ ?

Takes too long!





Learning Rate Schedule

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

learning rate as function of t:

$$\epsilon > 0$$

called *learning rate*

$$\epsilon(t) = \frac{\epsilon_0}{1 + \kappa t} \quad \text{decay rate } \kappa$$

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large the leap Δx is





Learning Rate Schedule

$$\left. \frac{dy}{dx} \right|_{x_0} \approx \frac{y(x_0 + \Delta x) - y(x_0 - \Delta x)}{2\Delta x}$$

$$x(t+1) = x(t) - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

learning rate as function of t:

$$\epsilon > 0$$

called *learning rate*

$$\epsilon(t) = \frac{\epsilon_0}{1 + \kappa t} \quad \text{decay rate } \kappa$$

$$\Delta x = - \epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large the leap Δx is

can also be a stepwise function (learning rate schedule)



Learning Rate Schedule

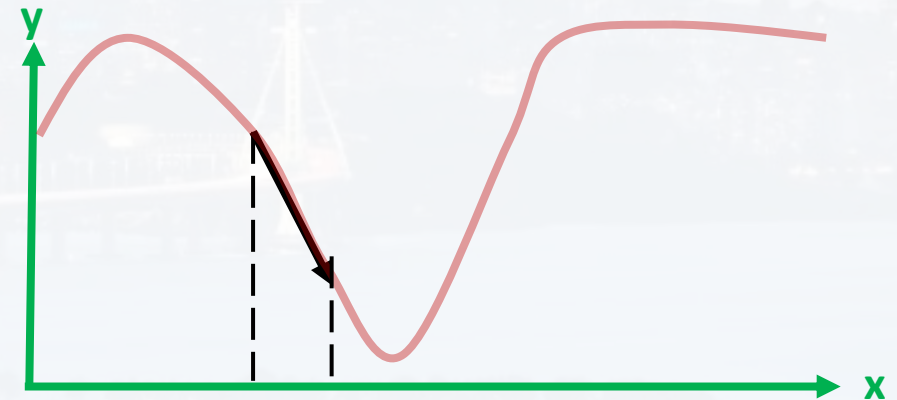
learning rate as function of t:

$$\epsilon(t) = \frac{\epsilon_0}{1 + \kappa t} \quad \text{decay rate } \kappa$$

$$\Delta x = -\epsilon \left. \frac{dy}{dx} \right|_{x(t)}$$

defines how large the leap Δx is

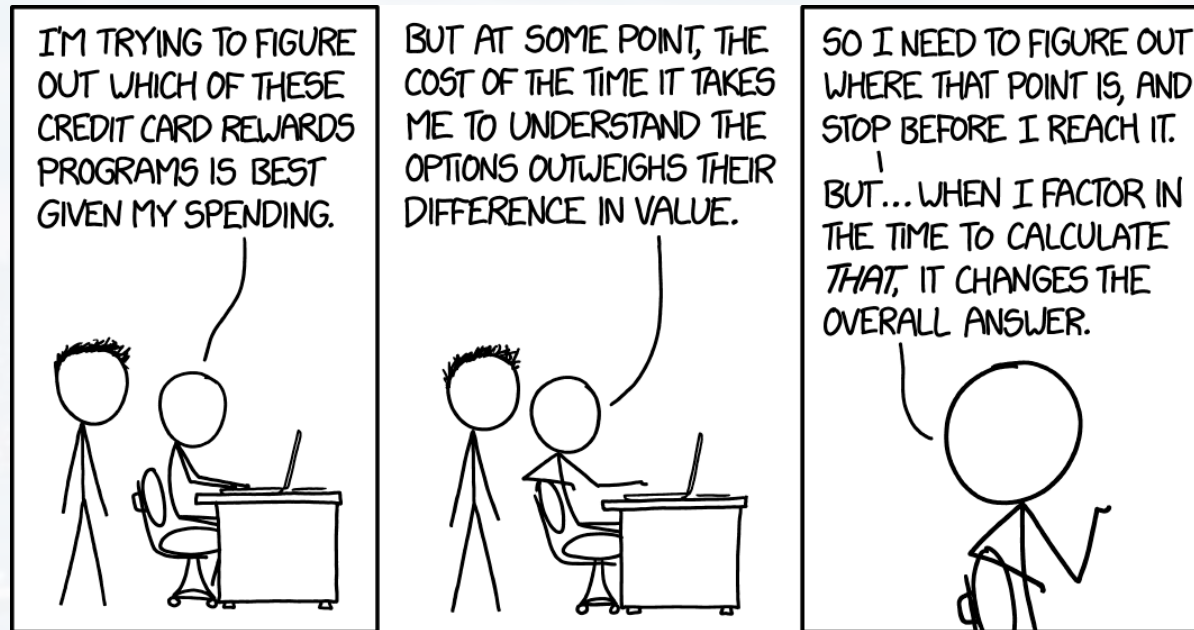
can also be a stepwise function (learning rate schedule)



better:
adaptive gradient, aka **AdaGrad**

$$r_{t+1} = r_t + \text{grad}(y)_x \circ \text{grad}(y)_x \quad \delta: \quad \text{small, } > 0$$

$$\epsilon \rightarrow \frac{\epsilon}{\sqrt{r_{t+1}} + \delta}$$



Outline

- The Problem

- **Gradient Descent**

- Vanilla

- Learning Rate Schedule

- Momentum

- L1 and L2

- More Finetuning

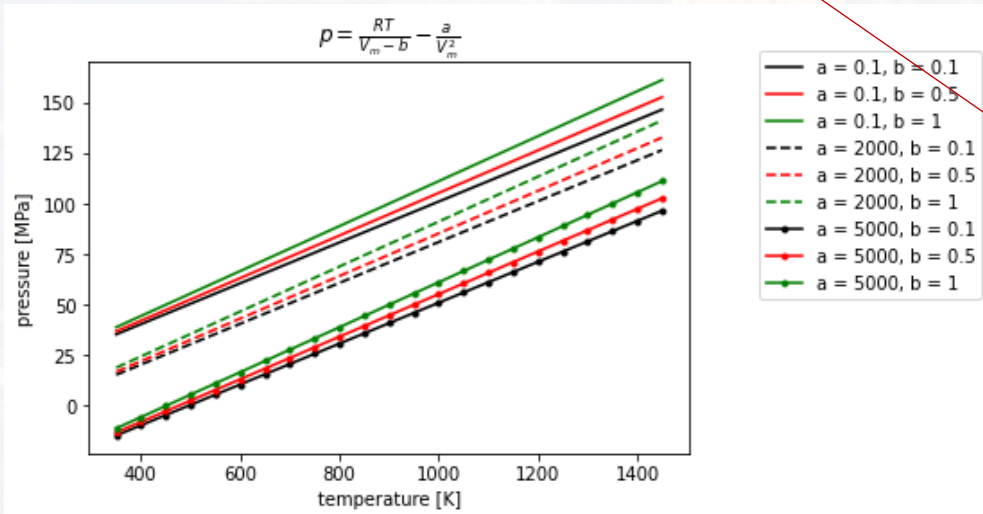
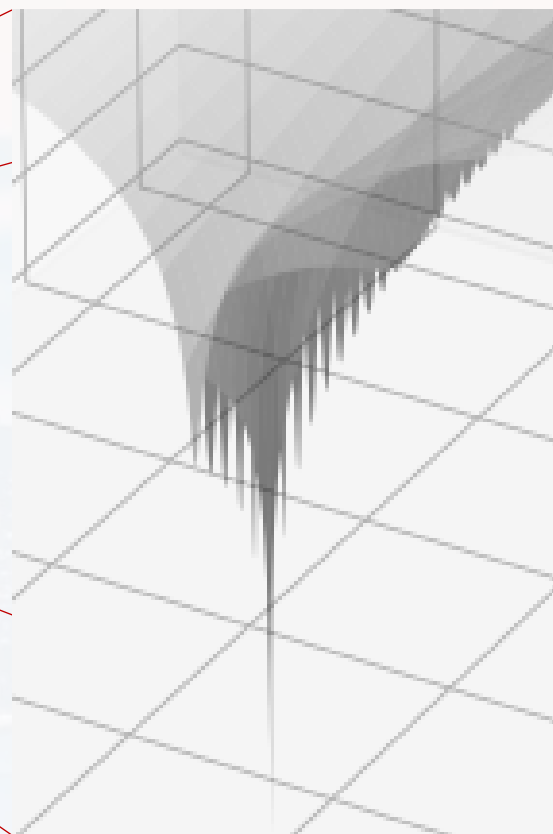
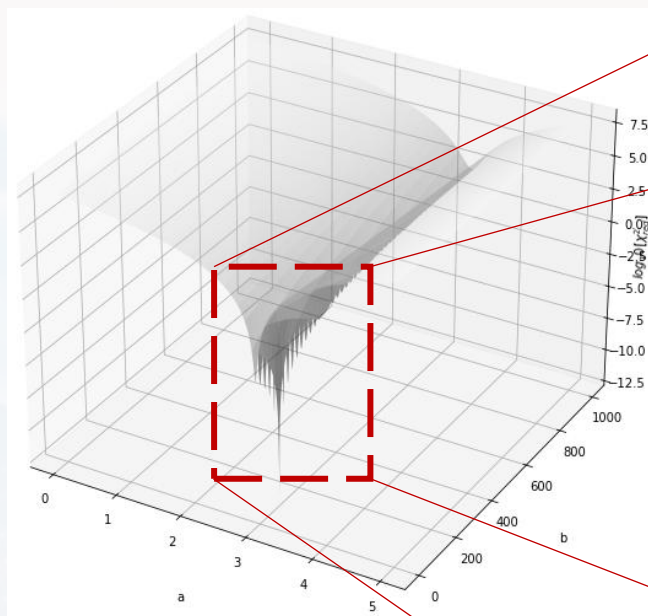
- Simulated Annealing



Momentum

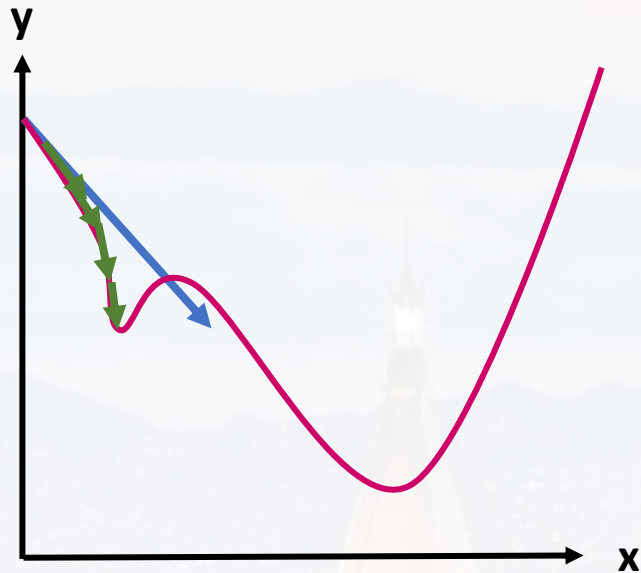
even with AdaGrad and learning rate schedule
→ would get stuck in local minimum

need to roll over → **momentum**





Momentum



taking the **average** of N previous gradients

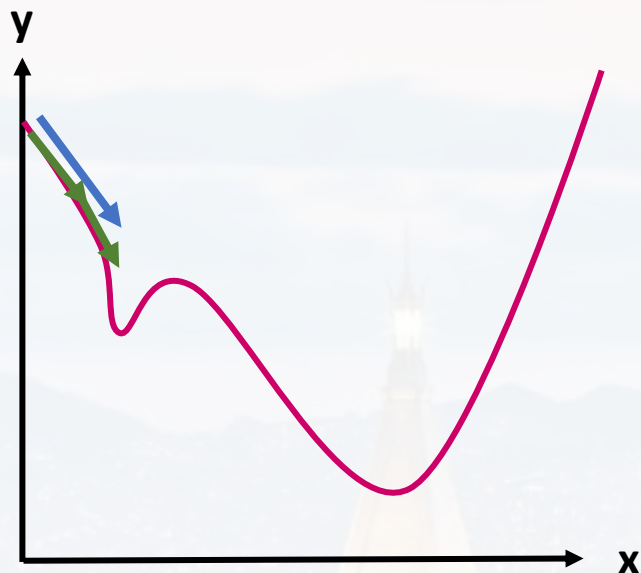
$$\langle grad(y)_{x(t)} \rangle = \frac{1}{N} [grad(y)_{x(t-1)} + grad(y)_{x(t-2)} + \dots + grad(y)_{x(t-N)}]$$

but we want more recent gradients to contribute more than older gradients

→ **weighted average** with weighting factor μ_k

$$\langle grad(y)_{x(t)} \rangle = \sum_{k=t-N}^{t-1} \mu_k \cdot grad(y)_{x(k)}$$

Finding a clever way to adjust μ_k during every iteration t



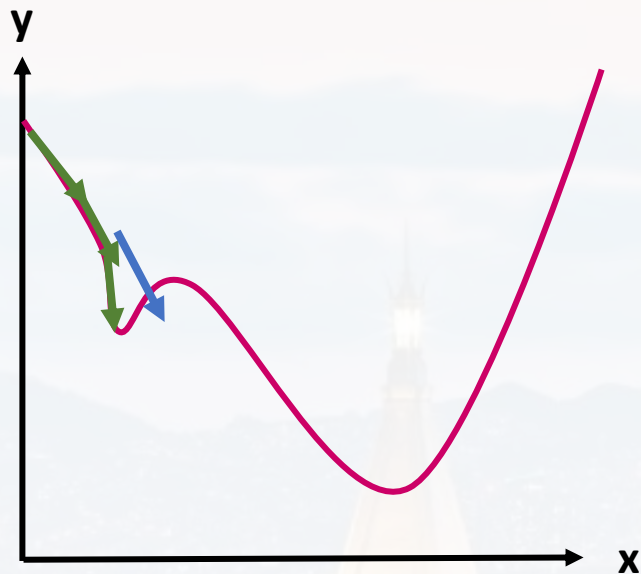
weighted average with weighting factor μ_k

Momentum

Finding a clever way to adjust μ_k during every iteration t

$$\langle grad(y)_{x(0)} \rangle = grad(y)_{x(0)} \quad \mu_0 = (0,1)$$

$$\langle grad(y)_{x(1)} \rangle = grad(y)_{x(1)} + \mu_0 \cdot grad(y)_{x(0)}$$



weighted average with weighting factor μ_k

Momentum

Finding a clever way to adjust μ_k during every iteration t

$$\langle grad(y)_{x(0)} \rangle = grad(y)_{x(0)} \quad \mu_0 = (0,1)$$

$$\langle grad(y)_{x(1)} \rangle = grad(y)_{x(1)} + \mu_0 \cdot grad(y)_{x(0)}$$

$$\langle grad(y)_{x(2)} \rangle = grad(y)_{x(2)} + \boxed{\mu_0} [grad(y)_{x(1)} + \boxed{\mu_0} grad(y)_{x(0)}]$$

$$\mu_{k=2} = \mu_0 \mu_0 = \mu_0^2$$

$$\langle grad(y)_{x(3)} \rangle = grad(y)_{x(3)} + \boxed{\mu_0} [grad(y)_{x(2)} + \boxed{\mu_0} [grad(y)_{x(1)} + \boxed{\mu_0} \cdot grad(y)_{x(0)}]]$$

... and so on...

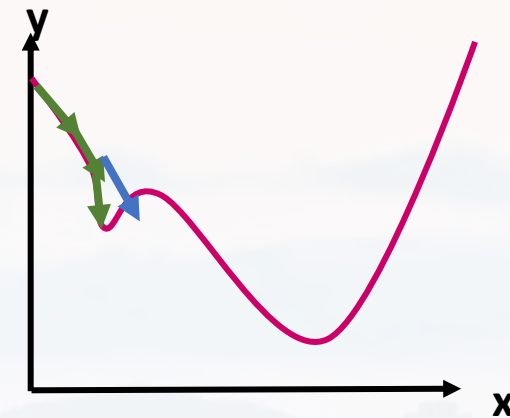


weighted average with weighting factor μ_k

$\mu_0 = (0,1)$ called "momentum"

$$\langle \text{grad}(\mathbf{y})_{x(3)} \rangle = \text{grad}(\mathbf{y})_{x(3)} +$$

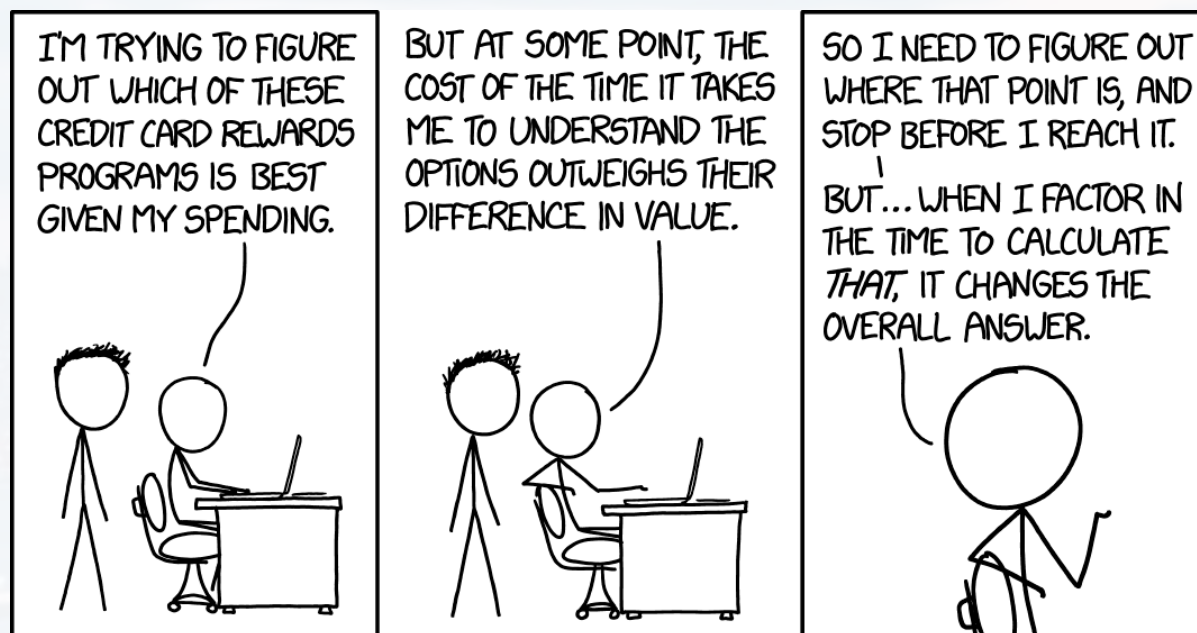
$$\mu_0 \left[\text{grad}(\mathbf{y})_{x(2)} + \mu_0 \left[\text{grad}(\mathbf{y})_{x(1)} + \mu_0 \cdot \text{grad}(\mathbf{y})_{x(0)} \right] \right] \quad \dots \text{and so on} \dots$$



Momentum

class Optimizer:

```
def __init__(self, learning_rate = 0.1, decay = 0, momentum = 0):  
    self.learning_rate = learning_rate  
    self.decay = decay  
    self.current_learning_rate = learning_rate  
    self.iterations = 0  
    self.momentum = momentum
```



Outline

- The Problem

- **Gradient Descent**

- Vanilla

- Learning Rate Schedule

- Momentum

- L1 and L2

- More Finetuning

- Simulated Annealing



L1 and L2

Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

Often, the extreme of the objective function is subject to **constraints**

sometimes we have some **prior knowledge** about the **independent variables**

recall: linear regression

finding best β by

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 \right\}$$

now:

constrain: **encourages sparsity of β**

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 + \lambda \|\beta\|^1 \right\}$$

λ Lagrangian Multiplier

called **L1 regularization**, or LASSO



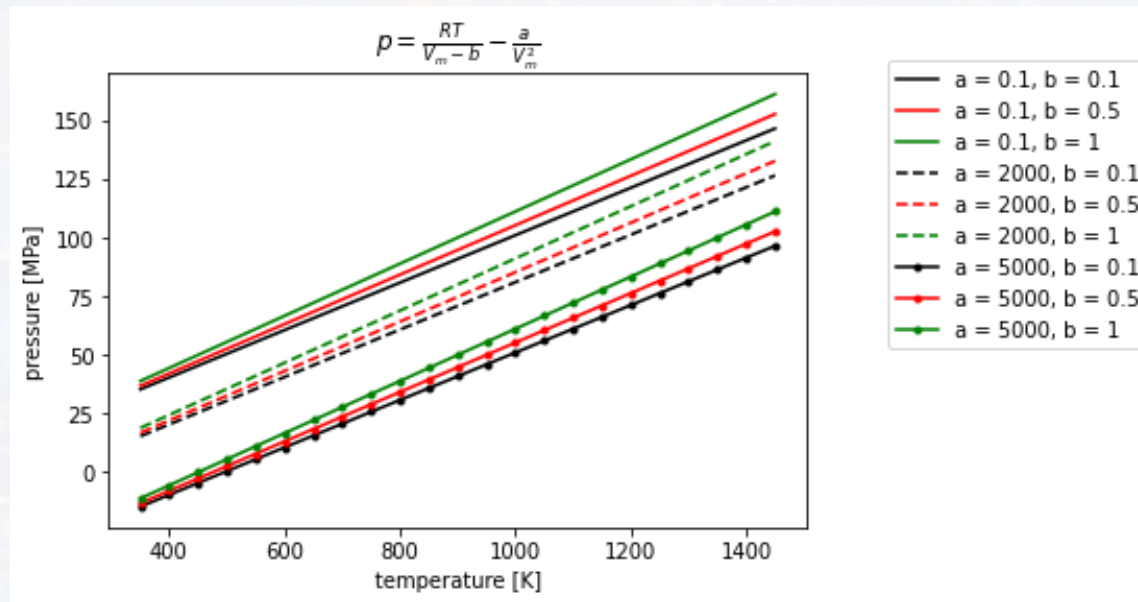
L1 and L2

Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

Often, the extreme of the objective function is subject to **constraints**

sometimes we have some **prior knowledge** about the **independent variables**

L1 regularization



We often have even hard constraints based on the laws of physics!



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

Often, the extreme of the objective function is subject to **constraints**

sometimes we have some **prior knowledge** about the **independent variables**

recall: linear regression

finding best β by

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 \right\}$$

now:

$$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T Y \longrightarrow$$

$$\min_{\beta} \left\{ \frac{1}{N} \|Y - X\beta\|^2 + \lambda \|\beta\|^2 \right\}$$

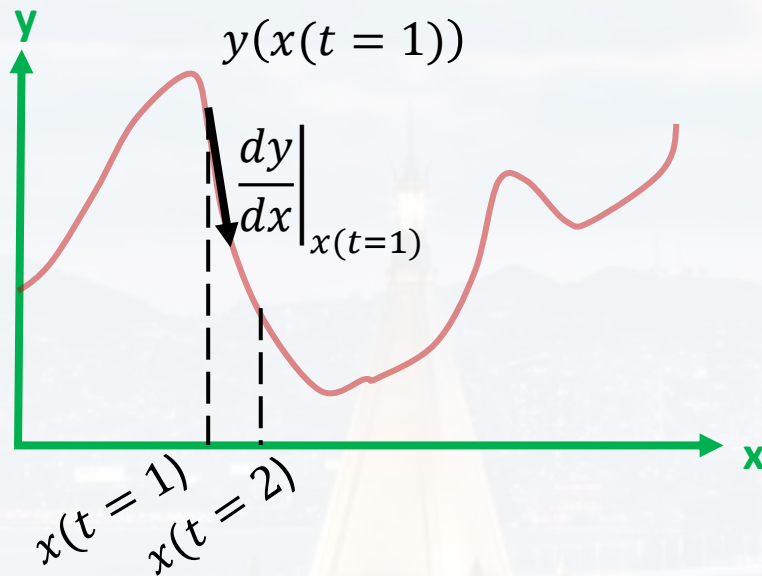
λ Lagrangian Multiplier

called **L2 regularization**, or RIDGE penalizes large β



L1 and L2 regularization

L1 and L2

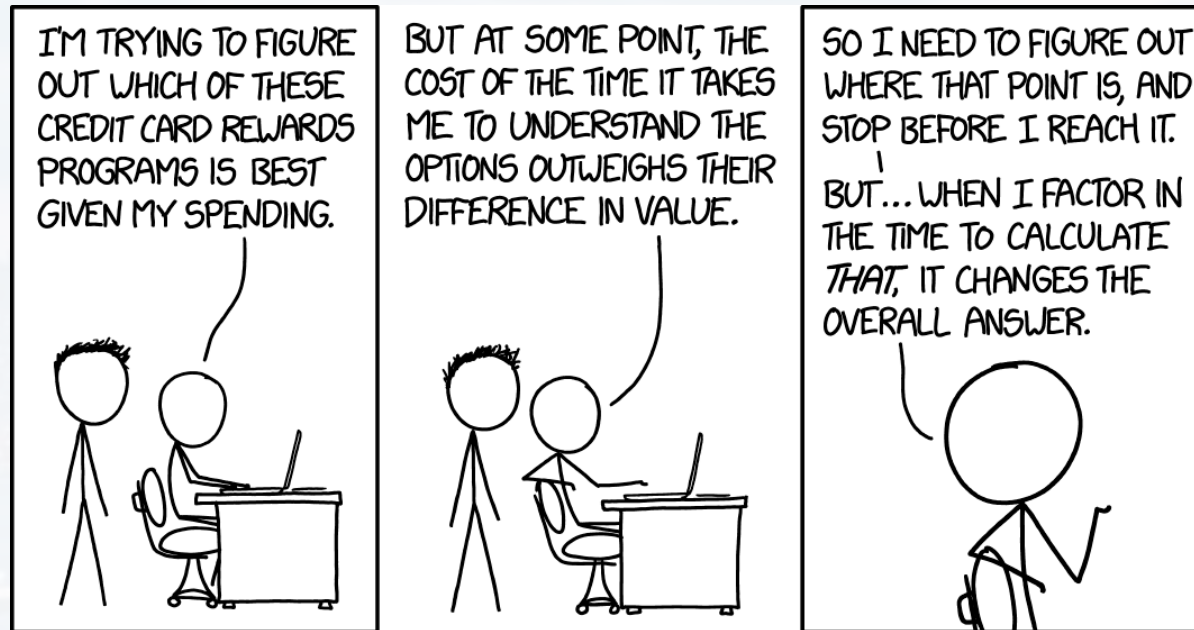


$$x(t=2) = x(t=1) - \varepsilon \frac{d[y + \lambda_1 \|x\|^1 + \lambda_2 \|x\|^2]}{dx} \Big|_{x(t=1)}$$

$$x(t=2) = x(t=1) - \varepsilon \frac{dy}{dx} \Big|_{x(t=1)}$$

$$- \varepsilon \frac{\lambda_1 d\|x\|^1}{dx} \Big|_{x(t=1)} - \varepsilon \frac{\lambda_2 d\|x\|^2}{dx} \Big|_{x(t=1)}$$

- gradient descent does not stop if values for x are too large and prefers sparsity
- note: the derivative of $\|x\|^1$ returns the sign (i. e. direction)
- usually $\lambda \ll \|x\|^n$
- will be important for ANNs later



Outline

- The Problem

- **Gradient Descent**

- Vanilla

- Learning Rate Schedule

- Momentum

- L1 and L2

- More Finetuning

- Simulated Annealing



Vanilla Gradient Descent → **S**tochastic **G**radient **D**escent

More Fine Tuning

Learning Rate Schedule, L1, L2

momentum

adaptive gradient, aka **AdaGrad**

$$\epsilon \rightarrow \frac{\epsilon}{\sqrt{r_{t+1}} + \delta}$$

Multiplying a decay factor to the sum of gradient squared (similar to momentum), aka **Root Mean Square Propagation** **RMSProp**

different scaling for all different directions

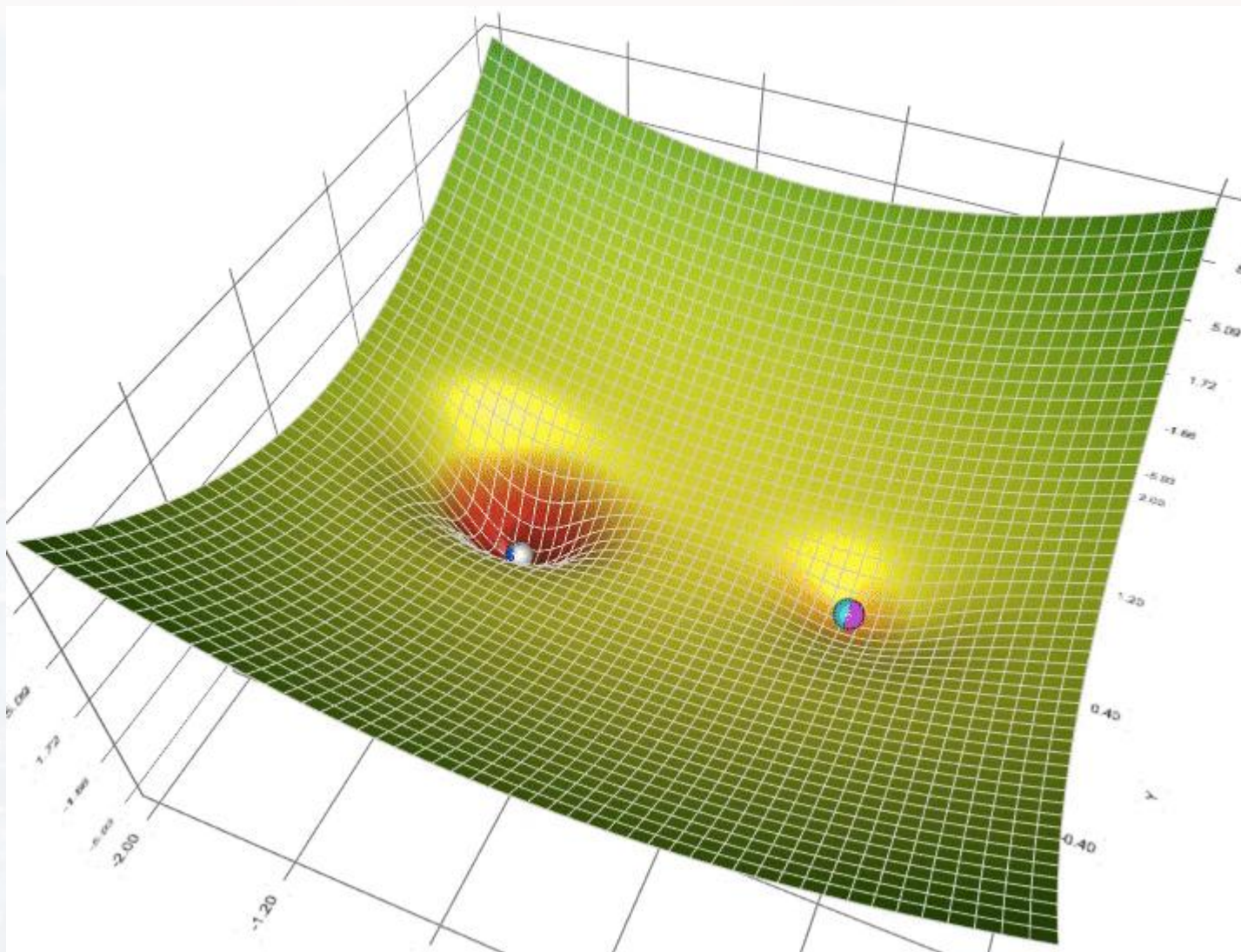
all combined:
Adaptive Moment Estimation
aka **Adam**



Lili Jiang

[TowardsDataScience](#)

More Fine Tuning



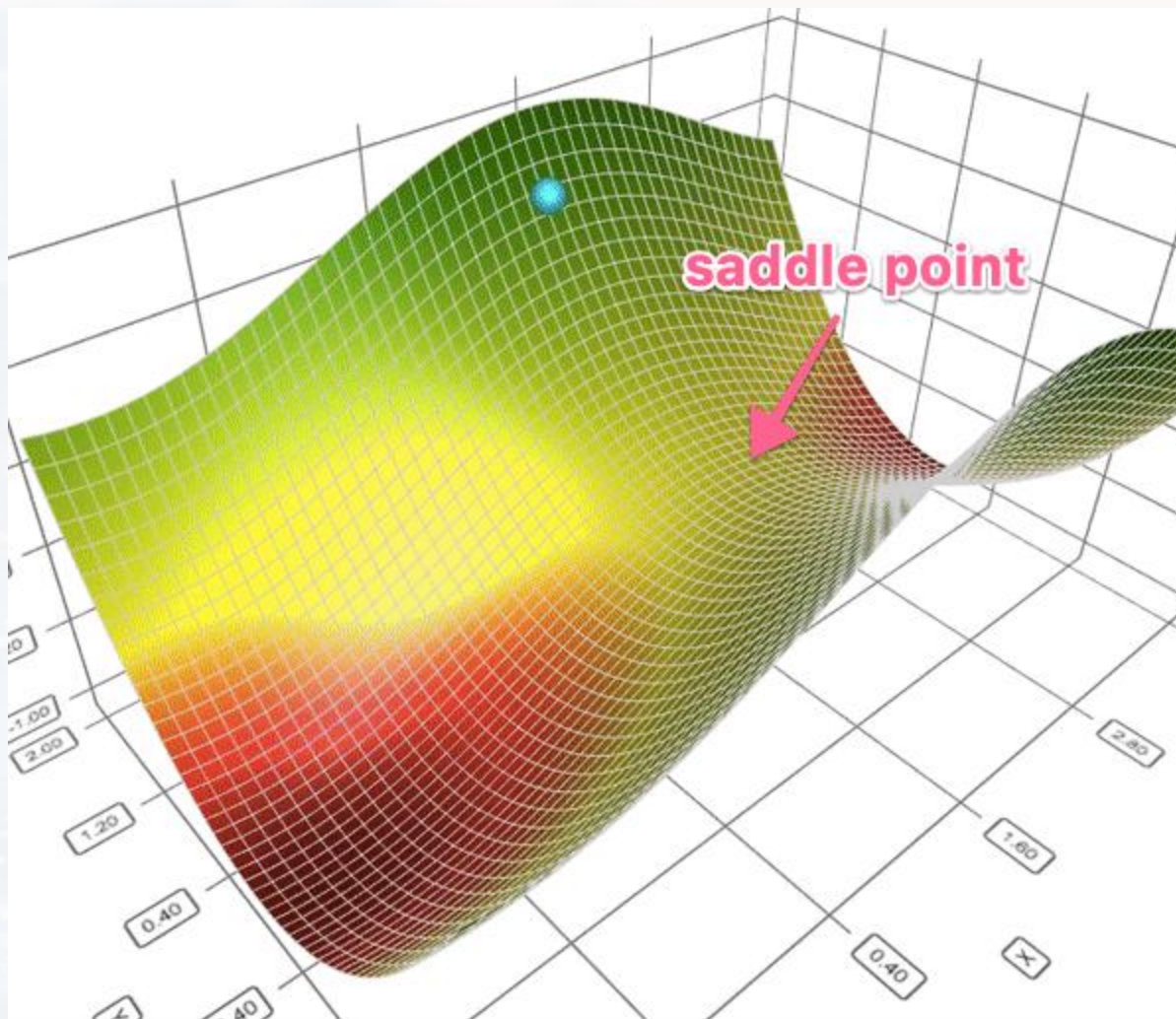
gradient descent (cyan),
momentum (magenta),
AdaGrad (white),
RMSProp (green),
Adam (blue)



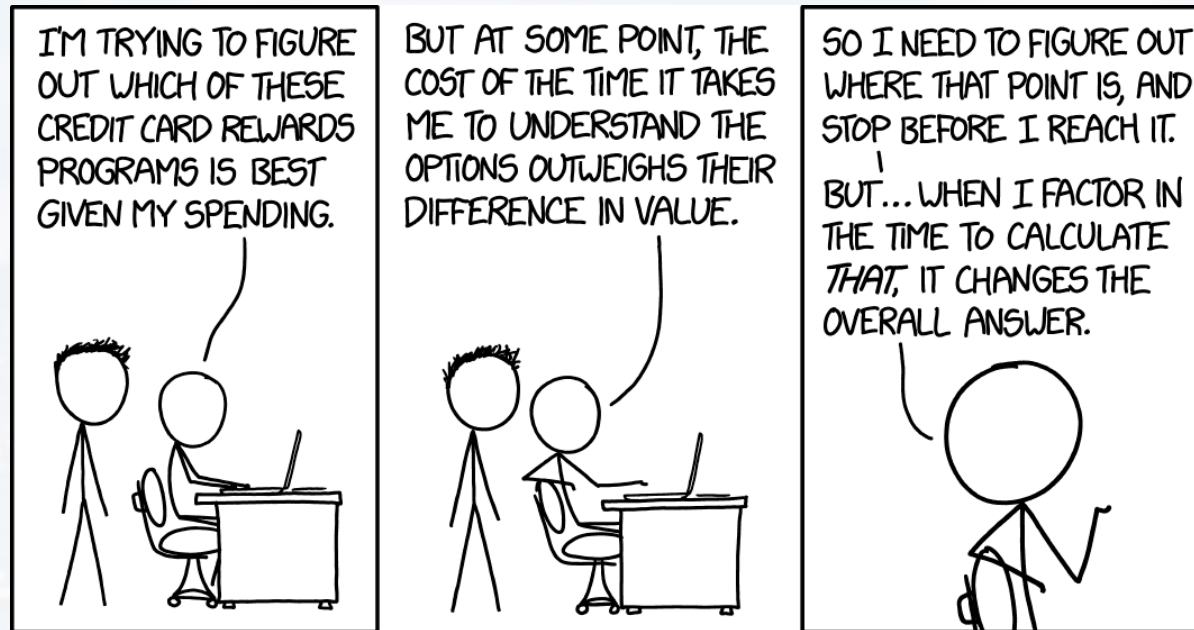
Lili Jiang

[TowardsDataScience](#)

More Fine Tuning



gradient descent (cyan),
momentum (magenta),
AdaGrad (white),
RMSProp (green),
Adam (blue)



Outline

- The Problem

- Gradient Descent

- Vanilla

- Learning Rate Schedule

- Momentum

- L1 and L2

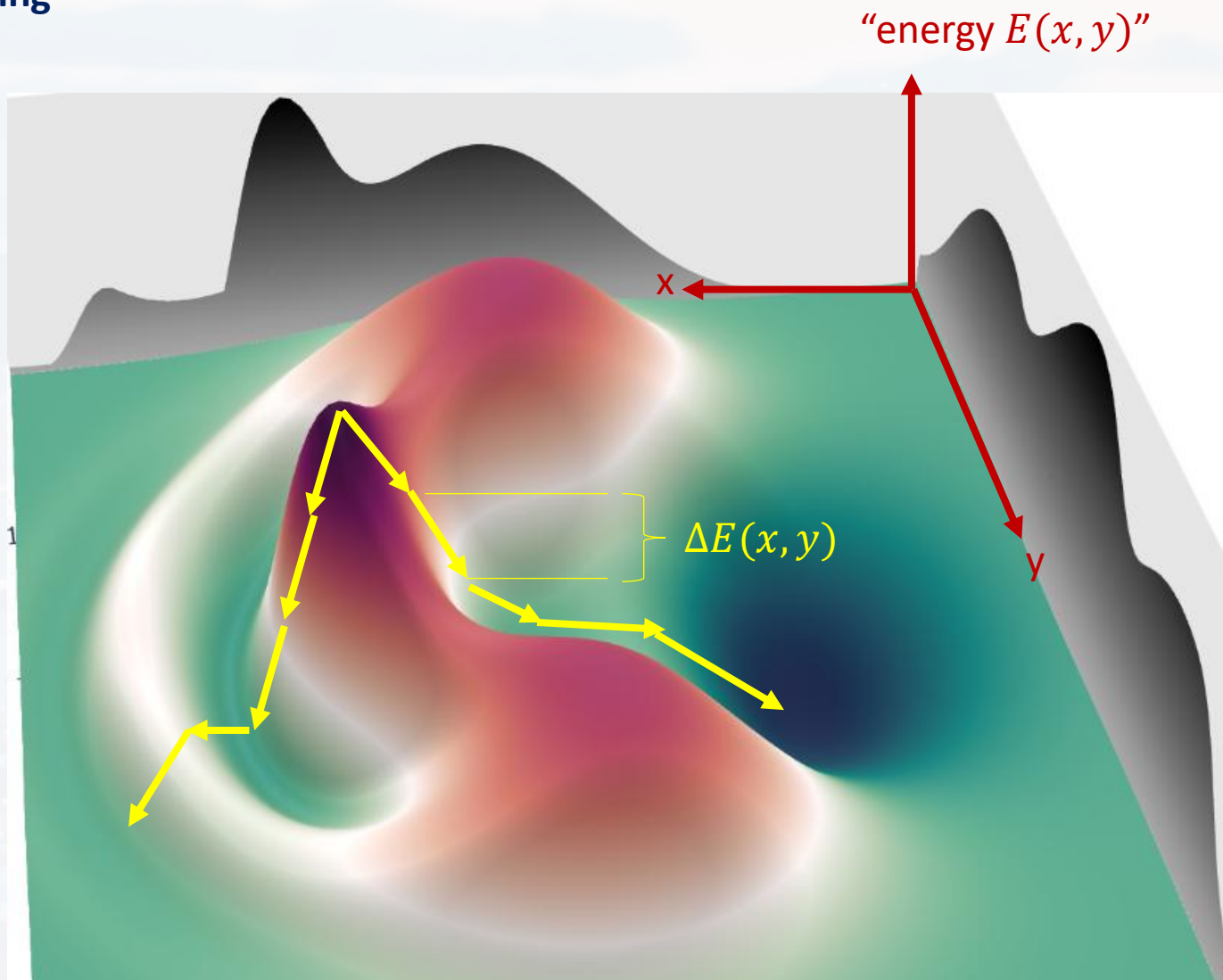
- More Finetuning

- **Simulated Annealing**



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

simulated annealing



If $\Delta E(x, y)$ is **negative**:
→ **always move**
(a ball always rolls down the hill)

If $\Delta E(x, y)$ is **positive**:
→ calculate the **probability to move**
→ leaves some chance to escape local minimum

T : temperature

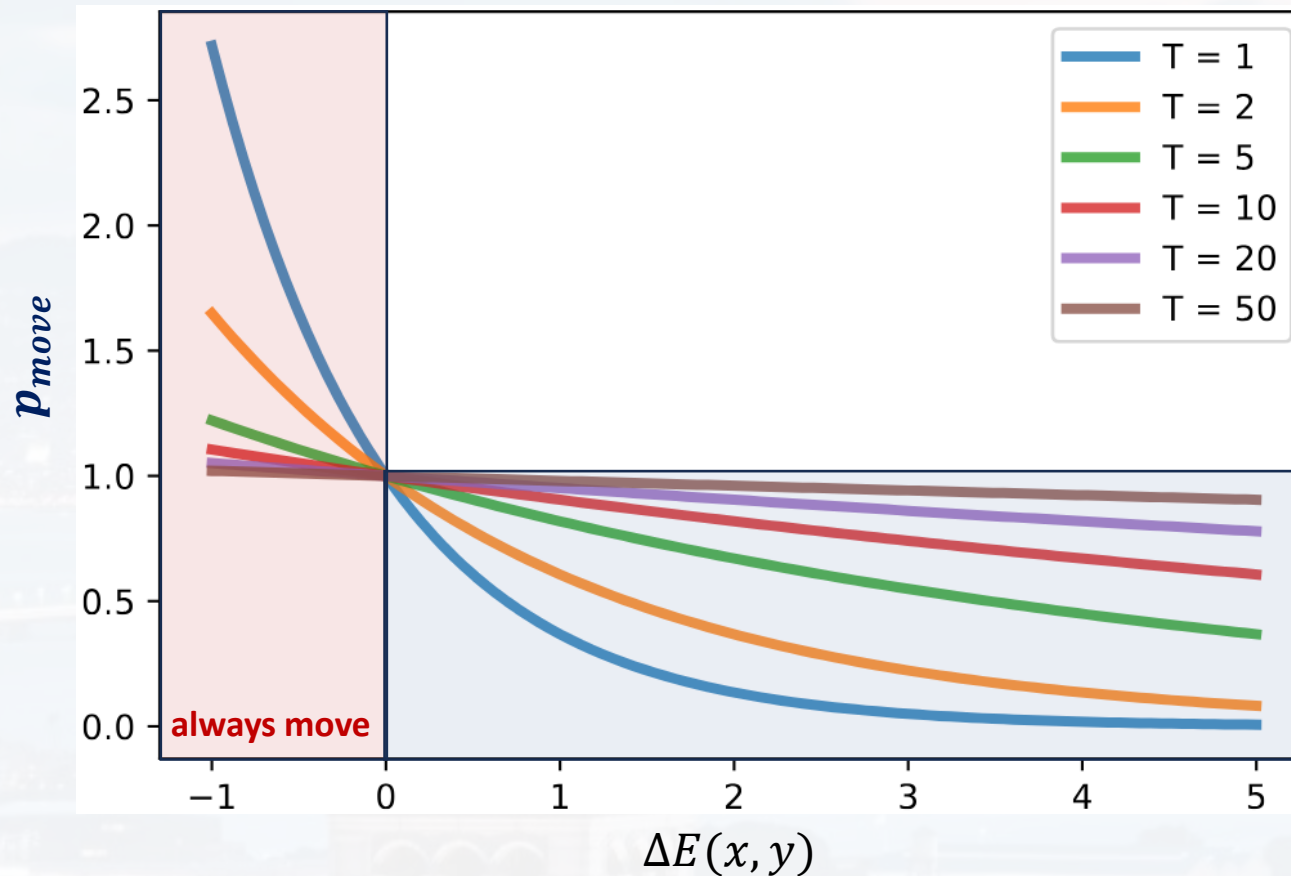
Boltzmann factor

$$p_{\text{move}} \sim \exp \left[-\frac{\Delta E(x, y)}{T} \right]$$



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

simulated annealing



If $\Delta E(x, y)$ is **negative**:
→ **always move**
(a ball always rolls down the hill)

If $\Delta E(x, y)$ is **positive**:
→ calculate the **probability to move**
→ leaves some chance to escape local minimum

T : temperature

Boltzmann factor

$$p_{move} \sim \exp \left[-\frac{\Delta E(x, y)}{T} \right]$$

slowly reducing $T \rightarrow$ making larger jumps ($\Delta E(x, y)$) less likely over time



Any algorithm needs a “goal” aka **objective function** that has to be **optimized** (finding an **extreme**)

simulated annealing

Metropolis (Chem 273):

- 1) suggest a random move Δx and Δy
- 2) calculate $\Delta E(x, y)$ based on Δx and Δy
- 3) move or not:
 - a) move if $\Delta E(x, y) < 0$
 - b) if $\Delta E(x, y) > 0$
 - draw a **random number** ρ from a **uniform distribution** in the interval $(0, 1)$
 - move if $\rho < \exp\left[-\frac{\Delta E(x, y)}{T}\right]$
- 4) reduce T and repeat

Thank you very much for your attention!

